How FP16 Vector Representation and TOAST Storage Layouts Shift Query Plan Selection in PostgreSQL: An Empirical and Cost-Model Study

Yash Pradhan
[Department / School Affiliation — Author Confirmation Required]
[Institution / University / Independent Affiliation — Author Confirmation Required]
[City, State/Province, Country — Author Confirmation Required]
Email: [Author Email — Confirmation Required] | ORCID: [Author ORCID — Confirmation Required]

Abstract*—PostgreSQL extensions such as pgvector rely on cost-based query optimization to choose between Hierarchical Navigable Small World (HNSW) index scans and sequential scans for top-*K* vector search. Because PostgreSQL's cost_seqscan prices only main-heap blocks (pg_class.relpages) and omits out-of-line TOAST (pg_toast) pages, pgvector 0.8.4 discounts HNSW startup cost in hnswcostestimate whenever estimated startup pages exceed relpages and the traversal ratio is below 0.5. We combine a source-level cost-model reconstruction of PostgreSQL 18.4 and pgvector 0.8.4 with benchmarks across three 768-dimensional representations, nine cost profiles, three primary scales (1,001, 5,000, 10,000 rows), and an 18-scale sweep (500–5,000 rows, 486 configurations). Under explicitly configured STORAGE EXTENDED, tested 32-bit vector(768) embeddings (fp32_extended) experienced zero compression reduction and moved to pg_toast, leaving 7 main-heap pages at 1,001 rows. Inline storage via 16-bit halfvec(768) (fp16_extended, 201 pages) or STORAGE PLAIN (fp32_plain, 501 pages) raises relpages above HNSW startup pages, deactivating the discount and leaving unadjusted startup costs (40.69× and 80.91× higher at Profile P4). Across the 486-configuration sweep, the ratio threshold activates between 786 and 787 rows on fp32_extended (reducing P4 HNSW startup cost by 88.06× displayed / 88.08× unrounded), 11/27 settings select sequential scans at 1,001 rows (2.90× median slowdown on fp32_extended at P7; benign on fp16_extended at P4), and all 27/27 select HNSW by 5,000 rows. End-to-end halfvec(768) cuts total storage by 55.21%–55.37% and index size by 49.90%–49.98%, achieving 0.985, 0.960, and 0.935 Recall@10 under P4 forced HNSW versus 0.985, 0.955, and 0.935 for fp32_extended (0.985, 0.955, and 0.930 for fp32_plain).

Keywords—PostgreSQL, pgvector, query optimization, HNSW indexing, TOAST storage, half-precision floating-point (FP16).

I. Introduction

Relational database management systems (RDBMSs) increasingly support vector similarity retrieval alongside scalar relational data for retrieval-augmented generation (RAG) and semantic search workloads [1], [2], [6], [7]. Rather than maintaining a separate vector store, deployments frequently integrate Hierarchical Navigable Small World (HNSW) graph indexes [5] directly into PostgreSQL through extensible access methods such as `pgvector` [2], [3] or PASE [6]. Inside a cost-based RDBMS, every top-*K* similarity query (`ORDER BY embedding <=> $1 LIMIT K`) is governed by the query optimizer, which estimates page I/O and CPU operator costs across competing access paths (`Index Scan` versus `Seq Scan` with a bounded top-*K* heap sort) before selecting an execution plan [4], [7], [24].

A fundamental cross-layer interaction arises because PostgreSQL's core sequential scan estimator (`cost_seqscan` in `costsize.c`) prices only the main-heap block count (`pg_class.relpages`) while omitting out-of-line pages stored in The Oversized-Attribute Storage Technique (`pg_toast`) [4], [21]. On standard 8 KiB (`8,192 B`) PostgreSQL pages, tuples exceeding the `2,032 B` TOAST threshold (`TOAST_TUPLE_THRESHOLD`, derived in Section II-A) are eligible for out-of-line storage [4]. In our benchmark, table provisioning explicitly configures `STORAGE EXTENDED` (which attempts inline lossless compression before externalizing oversized attributes) alongside `STORAGE PLAIN`. For our tested 768-dimensional 32-bit floating-point embeddings (`vector(768)`, `3,080 B` datum; `fp32_extended`), inline compression under `STORAGE EXTENDED` produced zero byte reduction on every row (`3,076.0 B` external `pg_column_size`), so PostgreSQL externalizes 100% of the vectors uncompressed into `pg_toast`. Consequently, the main table heap retains only compact 18-byte TOAST pointers (`varatt_external`) plus tuple headers (~157 tuples per full 8 KiB page), yielding a very small main-heap page count (`pg_class.relpages = 7` at N = 1,001 rows, `32` at N = 5,000, and `64` at N = 10,000).

To prevent `cost_seqscan`'s omission of `pg_toast` pages from making HNSW index scans appear artificially more expensive than sequential scans on TOASTed tables, `pgvector` 0.8.4 (`src/hnsw.c`, lines 187–194) implements an intentional TOAST-compensation heuristic in `hnswcostestimate` (`/* Adjust cost if needed since TOAST not included in seq scan cost */`) [3]. Whenever the estimated HNSW startup page count exceeds the main-heap page count and the estimated graph traversal ratio is below one-half (`startupPages > path->indexinfo->rel->pages && ratio < 0.5`), `hnswcostestimate` discounts the index startup cost by re-weighting `startupPages` from random to sequential page cost and subtracting the cost of pages beyond `relpages`. Crucially, however, `hnswcostestimate` keys this guard solely on main-heap `relpages` (`path->indexinfo->rel->pages`) rather than checking whether the vector column actually resides in `pg_toast`.

Consequently, two common physical design choices that keep 768-dimensional vectors inline within the main table heap alter how `hnswcostestimate` and `cost_seqscan` interact (Section II-A): (1) overriding the column storage policy to `STORAGE PLAIN` on single-precision vectors (`fp32_plain`, `3,080 B` datum, packing 2 tuples per 8 KiB page and expanding `relpages` to `501`, `2,500`, and `5,000`), or (2) representing coordinates as 16-bit half-precision floating-point values (`halfvec(768)` in `pgvector` [2], using IEEE-754 `binary16` format [25] under `STORAGE EXTENDED`, `fp16_extended`, `1,544 B` datum), which stays below the `2,032 B` TOAST threshold and packs 5 tuples per 8 KiB main-heap page (`relpages = 201`, `1,000`, and `2,000`). Moving vectors from out-of-line TOAST storage to inline main-heap storage raises `relpages` above `startupPages`, deactivating `pgvector`'s TOAST-compensation guard ($\Delta = 0$) and leaving

the unadjusted generic HNSW startup estimate while `cost_seqscan` prices the full inline heap.

A. Study Positioning and Research Questions

In the PostgreSQL and `pgvector` engineering communities, it is known that `cost_seqscan` ignores `pg_toast` pages [4], that `hnswcostestimate` includes a `relpages`-based adjustment to compensate for this omission [3], and that detoasting out-of-line FP32 vectors during sequential scans incurs multi-block buffer access overhead per row [21]. Peer-reviewed database studies have separately examined extensible ANN indexing in PostgreSQL [6], buffer-pool indirection and page-layout overheads relative to standalone Faiss [7], hybrid filtered vector execution [8], [9], cardinality estimation for vector range queries in analytical joins [10], and SSD-aware HNSW I/O [11]. Complementary to these works, this paper provides a closed-form C-source-to-`EXPLAIN` mathematical reconstruction and a controlled 18-scale empirical characterization showing how `pgvector`'s `relpages`-keyed guard (`startupPages > relpages && ratio < 0.5`), PostgreSQL's `cost_seqscan` and fuzzy path comparator (`compare_path_costs_fuzzily`), and inline storage via `halfvec(768)` or `STORAGE PLAIN` jointly govern HNSW versus `Seq Scan` crossover intervals across optimizer cost profiles.

We investigate three specific research questions:

- **RQ1 (Cost-Model Mechanics, Guard Regions, and 18-Scale Crossover Intervals):** How do vector representation (`vector(768)` vs. `halfvec(768)`) and column storage policy (`STORAGE EXTENDED` vs. `STORAGE PLAIN`) interact with `pgvector` 0.8.4's `hnswcostestimate` and PostgreSQL 18.4's `LIMIT` and fuzzy cost-comparison model across nine planner cost profiles (`P1`–`P9`), three primary benchmark scales ($N \in \{1{,}001, 5{,}000, 10{,}000\}$), and a verified 18-scale plan-only sweep ($N \in [500, 5{,}000]$, `486` configurations)?
- **RQ2 (Physical Storage, Shared-Buffer Traffic, and Client Round-Trip Latency):** What are the resulting trade-offs in physical relation and index byte footprints, root-node 8 KiB shared-buffer block accesses (including correlation with PostgreSQL's `BAS_BULKREAD` threshold), and client-observed query round-trip latency (P_{50}, P_{95}, outlier sensitivity, and non-parametric query-level inference)?
- **RQ3 (Retrieval Agreement and Boundary Tie Semantics):** How does primary conventional `Recall@10` relative to the exhaustive exact FP32 reference set behave across exact sequential scans and forced HNSW traversal, and how do exact duplicate-vector equivalence classes differ from genuine ϵ-tolerance near-ties?

B. Summary of Contributions and Scope Boundaries

1. **Closed-Form Cost-Model Reconstruction and Full 486-Cell Candidate Validation:** We reconstruct the six-step execution sequence in `pgvector` 0.8.4 (`hnswcostestimate` in `src/hnsw.c` [3]) and its three-level propagation through PostgreSQL 18.4 (`genericcostestimate`, `cost_index`, `index_pages_fetched`, `cost_seqscan`, `cost_tuplesort`, `create_limit_path`, and `compare_path_costs_fuzzily` [4], [22]). We validate our closed-form `Seq Scan + Top-k Sort` equation against live forced `exact`-mode `EXPLAIN` captures across all 486 configurations (18 × 3 × 9), matching PostgreSQL's displayed startup and total costs with `0.0000` maximum difference (i.e., identical at two-decimal `EXPLAIN` precision).

2. **18-Scale Planner Crossover Characterization:** Using the distinct `(scale, representation, profile)` setting (27 per scale; `486` across the 18-scale sweep) as the primary unit for planner decisions, we separate the analytical `hnswcostestimate` guard boundary from the actual optimizer crossover. For $m = 16$, $ef_{\text{search}} = 40$, `ratio < 0.5` activates between $N = 786$ (`0.500041`) and $N = 787$ (`0.499493`), reducing `fp32_extended` Profile P4 HNSW startup cost by `88.06×` in displayed `EXPLAIN` units (`788.15 -> 8.95`; `88.08×` unrounded) on a same-scale basis (`87.93×` display / `87.95×` unrounded across $N = 786 \rightarrow 787$) and reducing root `Limit` total cost from `797.45` to `29.31`, flipping `7/9` profiles of `fp32_extended` to `Index Scan`. Inline `fp32_plain` and `fp16_extended` never activate the guard (`0/162` sweep cells each) and instead cross over to HNSW across $N \in [500, 5{,}000]$ as linear `Seq Scan + Sort` cost overtakes logarithmic unadjusted HNSW cost (including a `1% STD_FUZZ_FACTOR` total-cost tie-break at $N = 850$, `fp32_plain`, `P6`, where HNSW root `Limit` total cost `846.04` and `Seq Scan + Sort` total cost `843.86` lie within `1%` and HNSW wins on lower startup cost `814.52 < 843.36`, below the `1%` fuzzy startup threshold `843.36 / 1.01 = 835.01`).

3. **Storage, Buffer, Latency, and Retrieval Characterization:** We show that `fp16_extended` cuts total storage by `55.21%–55.37%` and HNSW index size by `49.90%–49.98%`, avoids `fp32_extended`'s `relpages + 3N` sequential-scan TOAST buffer amplification, and stays below PostgreSQL's `NBuffers / 4 = 4,096` block `BAS_BULKREAD` threshold at $N = 10{,}000$. Crucially, inline storage does not automatically induce a harmful plan: at $N = 1{,}001$, `fp16_extended`'s Profile `P4 Seq Scan` choice has no statistically significant client-latency penalty (`1.674 ms` vs. `1.591 ms` P_{50}, $p_{\text{adj}} = 0.2024$), whereas the physically harmful tested fallback is TOASTed `fp32_extended` at `P7` (`2.90×` client median slowdown: `4.935 ms` vs. `1.701 ms`). Separating HNSW index execution advantages from FP16 representation effects, forced HNSW index scans deliver large client-latency speedups over exact sequential scans at $N \geq 5{,}000$ across all three representations (`15.76×` and `33.86×` median speedups on `fp32_extended`, `3.31×` and `18.59×` on `fp32_plain`, and `2.29×` and `5.15×` on `fp16_extended`, where the smaller ratio on `fp16_extended` reflects its faster inline exact scan). Under exact scans, `fp16_extended` matches `599/600` top-10 FP32 reference slots (`1.000 / 0.995 / 1.000` conventional `Recall@10`, with a 1-slot swap on `Q17` at $N = 5{,}000$), and under `P4 forced_hnsw` it achieves `0.985 / 0.960 / 0.935` conventional `Recall@10` versus `0.985 / 0.955 / 0.935` on `fp32_extended` and `0.985 / 0.955 / 0.930` on `fp32_plain` (`0.9737` and `0.9684` at $N = 10{,}000$ excluding a 28-chunk exact vector-equality cluster on `Q9`, detailed in Appendix A).

Scope Boundaries: This is an observational study of unmodified PostgreSQL `18.4` and `pgvector 0.8.4` over unfiltered single-worker (`max_parallel_workers_per_gather = 0`) top-10 cosine queries ($d = 768$, $m = 16$, $ef_{\text{construction}} = 64$, $ef_{\text{search}} = 40$) using a prepared `BeIR/msmarco` collection [14], [15] and 20 fixed topical queries (`q0`–`q19`) in a warm-cache regime. Because no human relevance judgments (`qrels`) are used, `Recall@10` measures candidate agreement with exact FP32 cosine ranking rather than judged IR relevance.

II. Background and Source-Level Cost-Model Analysis

A. PostgreSQL Tuple Storage, `TOAST_TUPLE_THRESHOLD = 2,032 B`, and `relpages`

PostgreSQL stores relation data in 8 KiB (`BLCKSZ = 8,192 B`) heap pages, each with a 24-byte `PageHeaderData` and 4-byte `ItemIdData` line pointers [4]. In `src/include/access/heaptoast.h` and `htup_details.h`, PostgreSQL defines the compile-time tuple-level TOAST threshold by requiring at least `TOAST_TUPLES_PER_PAGE = 4` tuples per page, and sets the default relation-level `toast_tuple_target` parameter to `TOAST_TUPLE_TARGET = TOAST_TUPLE_THRESHOLD`:

$$\text{TOAST_TUPLE_THRESHOLD} = \text{MaximumBytesPerTuple}(4) = \text{MAXALIGN_DOWN}(\lfloor(8192 - \text{MAXALIGN}(24 + 4 \times 4)) / 4\rfloor) = \mathbf{2{,}032\ B} \quad (1)$$

For a table that does not override `toast_tuple_target`, this `2,032 B` threshold is compared against the uncompressed heap tuple length `t_len`. In our two-column experimental schema (`CREATE TABLE chunks_{scale}_{rep} (chunk_id INT PRIMARY KEY, embedding {vector(768)|halfvec(768)} NOT NULL)` in the provisioning DDL `[Supplementary Material – Repository/DOI Placeholder]`), both columns are `NOT NULL` (`hasnull = false`), so `SizeofHeapTupleHeader` (`23 B`) aligns to `t_hoff = 24 B`, `chunk_id` occupies `4 B` (bytes `24..27`), and under effective 4-byte attribute alignment (`typalign = 'i'`) the `embedding` datum starts at byte offset `28` (`t_len = 28 + VARSIZE`), while 8-byte page-slot alignment (`MAXALIGN(t_len)`) adds `4 B` of trailing padding (`32 B` combined header/key/page-alignment overhead before adding the `4 B ItemIdData` line pointer) [4]:

1. **`vector(768)` under `STORAGE PLAIN` (`fp32_plain`):** A 768-D IEEE-754 `binary32` vector [2], [25] occupies 768 × 4 B = 3,072 B plus a 4-byte `varlena` header (`VARHDRSZ = 4`), 2-byte `dim`, and 2-byte `unused` field, totaling VARSIZE = 4d + 8 = 3,080 B (`pg_column_size = 3,080 B`). This yields a raw heap tuple length of t_{len} = 24 + 4 + 3,080 = 3,108 B and an 8-byte aligned page-slot footprint of MAXALIGN(t_{len}) = 3,112 B (plus a `4 B ItemIdData` line pointer = `3,116 B` per page slot). Setting `STORAGE PLAIN` disables compression and out-of-line TOAST chunk storage (`toast_table_bytes = 0 B`, `0` chunk pages), packing **2 tuples per 8 KiB page** (`relpages = 501`, `2,500`, and `5,000` at N = 1,001, 5,000, and 10,000). Because our table-provisioning workflow (`[Supplementary Material – Repository/DOI Placeholder]`) executes `CREATE TABLE` before executing `ALTER TABLE ... ALTER COLUMN embedding SET STORAGE PLAIN`, PostgreSQL allocates an associated `pg_toast` relation and its mandatory 1-page B-tree index metapage (`toast_index_bytes = 8,192 B`) at `CREATE TABLE` time when `embedding` inherits its type-level TOAST-eligible storage policy (`STORAGE EXTERNAL` [2]); the subsequent `SET STORAGE PLAIN` statement prevents `INSERT` from writing TOAST chunks (`toast_table_bytes = 0 B`) without dropping the already-created `pg_toast` relation, yielding toast_total_bytes = toast_table_bytes + toast_index_bytes = 0 + 8,192 = 8,192 B (1 page).

2. **`vector(768)` under `STORAGE EXTENDED` (`fp32_extended`):** Our provisioning workflow explicitly sets `ALTER COLUMN embedding SET STORAGE EXTENDED` (PostgreSQL's general `varlena` default, whereas `pgvector` 0.8.4's type-level storage policy is `STORAGE EXTERNAL` [2]). Because t_{len} = 3,108 B > 2,032 B, `toast_tuple_try_compression` attempts inline lossless compression (`pglz` / `lz4`); across our tested corpus (`[Supplementary Material – Repository/DOI Placeholder]`), compression produced zero byte reduction on any row (`min = max = avg_col_size_bytes = 3,076.0 B`). Consequently, `toast_save_datum` (`src/backend/access/common/toast_internals.c`) externalizes every attribute uncompressed to `pg_toast` as two chunks per vector (packing 4 chunks, or 2 vectors, per 8 KiB TOAST chunk page) and replaces the main-heap attribute with an 18-byte external TOAST pointer (`varatt_external`). At N = 1,001, the external TOAST chunk heap occupies toast_table_bytes = 4,104,192 B (`501` pages) and its B-tree chunk index occupies toast_index_bytes = 65,536 B (`8` pages), producing toast_total_bytes = toast_table_bytes + toast_index_bytes = 4,169,728 B (`509` pages). When writing an uncompressed external datum of size `VARSIZE = 3,080 B`, `toast_save_datum` strips the 4-byte `VARHDRSZ` header and stores `VARSIZE - VARHDRSZ = 3,076 B` in `toast_pointer.va_extinfo`, which `pg_column_size()` (`dbsize.c`) returns for on-disk TOAST pointers (`3,076 B`). In the main heap, compact pointer tuples (`t_len = 46 B`, `MAXALIGN = 48 B`, plus `4 B ItemIdData` = `52 B` per page slot) pack at **approximately 157 tuples per full 8 KiB page** across our 18-scale corpus (`relpages = 7` at N = 1,001 [6 full pages + 1 partial page], `32` at N = 5,000, and `64` at N = 10,000).

3. **`halfvec(768)` under `STORAGE EXTENDED` (`fp16_extended`):** Storing each coordinate in IEEE-754 `binary16` format [25] via `pgvector`'s `halfvec(768)` type [2] reduces the datum size (`pg_column_size`) to 2d + 8 = 1,544 B, the raw heap tuple length to t_{len} = 24 + 4 + 1,544 = 1,572 B, and the 8-byte aligned page-slot footprint to MAXALIGN(t_{len}) = 1,576 B (plus a `4 B ItemIdData` line pointer = `1,580 B` per page slot). Because t_{len} = 1,572 B ≤ 2,032 B (and MAXALIGN(t_{len}) = 1,576 B ≤ 2,032 B), PostgreSQL never invokes TOAST compression or out-of-line chunk storage (`toast_table_bytes = 0 B`, `0` chunk pages, identical under `STORAGE EXTENDED` and `STORAGE EXTERNAL`), while the `pg_toast` relation allocated at `CREATE TABLE` time retains only its 1-page B-tree index metapage (`toast_index_bytes = 8,192 B`), giving toast_total_bytes = 0 + 8,192 = 8,192 B. Since 5 × 1,580 B = 7,900 B ≤ 8,168 B, `fp16_extended` packs **5 tuples per 8 KiB main-heap page** (`relpages = 201`, `1,000`, and `2,000` at N = 1,001, 5,000, and 10,000).

PostgreSQL's `ANALYZE` populates `pg_class.relpages` (`path->indexinfo->rel->pages`) strictly with main-heap blocks, omitting `pg_toast` [4].

B. Six-Step Execution Sequence in `pgvector` 0.8.4 `hnswcostestimate`

Reconciling `pgvector` 0.8.4 (`src/hnsw.c` [3]) and PostgreSQL 18.4 (`src/backend/utils/adt/selfuncs.c` [4]) against our captured `EXPLAIN` plan trees establishes the six-step cost computation for an unfiltered top-K query (`ORDER BY embedding <=> $1 LIMIT 10`):

Step 1 — Zero-Initialization: `hnswcostestimate` zeroes `GenericCosts costs` (`MemSet(&costs, 0, sizeof(costs))`), leaving `costs.numIndexTuples = 0.0`.

Step 2 — Generic Index Estimate (`genericcostestimate`): Because `costs.numIndexTuples == 0` and `path->indexclauses == NIL`, `genericcostestimate` sets `*indexSelectivity = 1.0`, clamps `costs.numIndexTuples` to `index->tuples = N`, sets `costs.numIndexPages = index->pages`, and computes the full-index scan cost (`cpu_index_tuple_cost = 0.005`):

$$\text{costs.indexTotalCost} = \text{numIndexPages} \cdot \text{spc_random_page_cost} + N \cdot (\text{cpu_index_tuple_cost} + \text{cpu_operator_cost}) \quad (2)$$

At N = 1,001, Profile P4 (`random_page_cost = 4.0`, `seq_page_cost = 1.0`, `cpu_operator_cost = 0.0025`, `cpu_tuple_cost = 0.01`), the full-index CPU term is 1001 × 0.0075 = 7.5075. With `numIndexPages = 499` for FP32 (`1996.0` page I/O cost) and `250` for FP16 (`1000.0` page I/O cost), (2) gives $\text{indexTotalCost}_{\text{FP32}}$ = 1996.0 + 7.5075 = 2003.5075 (**`2003.51`**) and $\text{indexTotalCost}_{\text{FP16}}$ = 1000.0 + 7.5075 = 1007.5075 (**`1007.51`**).

Step 3 — HNSW Traversal Selectivity (`ratio`): `hnswcostestimate` models the fraction of index tuples visited during greedy upper-layer descent ($\text{entryLevel}(N) = \lfloor \ln N / \ln m \rfloor$, which equals 2 for $256 \le N < 4{,}096$ and 3 for $4{,}096 \le N < 65{,}536$ at $m = 16$) and layer-0 beam search (`hnsw.ef_search = 40`):

$$\text{candidateTuples}(N) = \text{entryLevel}(N) \cdot m + C_{\text{cal}}(m, ef_{\text{search}}) \cdot \ln N,\quad \text{ratio}(N) = \text{candidateTuples}(N) / N \tag{3}$$

For $m = 16$, $ef_{\text{search}} = 40$, the empirically calibrated layer-0 scaling coefficient $C_{\text{cal}}(16, 40)$ = 54.152441134 (`LAYER0_TUPLE_SCALING_CONST`, calibrated from captured `EXPLAIN` plans at N = 1,001) reproduces the observed unadjusted startup costs across all 18 scales ($N \in [500, 5{,}000]$) to within ≤ 0.01 cost units. Notice that ratio(N) depends only on N, m, and ef_{search} (not on dimension d or storage policy) and crosses `0.50` between ***N* = 786 (ratio = 0.500041 ≥ 0.50)** and ***N* = 787 (ratio = 0.499493 < 0.50)**; at N = 1,001, 5,000, and 10,000, ratio(N) equals **`0.405720`**, **`0.101845`**, and **`0.054676`**.

Step 4 — Unadjusted Generic HNSW Startup Estimate: `hnswcostestimate` scales `costs.indexTotalCost` by `ratio`:

$$\text{indexStartupCost}_{\text{initial}} = \text{costs.indexTotalCost} \cdot \text{ratio} = (\text{numIndexPages} \cdot \text{spc_random_page_cost}) \cdot \text{ratio} + \text{indexCPUCost}_{\text{total}} \cdot \text{ratio} \tag{4}$$

At Profile P4 (N = 1,001), using the 8-decimal calibrated ratio $\text{ratio}_{\text{cal}}$ = 0.40572022 (`0.40572021`63 full precision, where the startup CPU term is 7.5075 × 0.40572022 = 3.045945 ≈ 3.0459), (4) evaluates to:

$\text{indexStartupCost}_{\text{initial, FP32}}$ = 1996.0 × 0.40572022 + 3.045945 = 809.817559 + 3.045945 = **812.8635** (**812.86** in `EXPLAIN`)

$\text{indexStartupCost}_{\text{initial, FP16}}$ = 1000.0 × 0.40572022 + 3.045945 = 405.720220 + 3.045945 = **408.7662** (**408.77** in `EXPLAIN`)

(Note: Truncating `ratio` to 6 decimal places as `0.405720` with 4-decimal intermediate rounding `startupPages = 202.4543` and `101.4300` and CPU term `3.0460` [Supplementary Material — Repository/DOI Placeholder] gives 809.8172 + 3.0460 = 812.8632 and 405.7200 + 3.0460 = 408.7660; both parameterizations round to the exact same two-decimal captured `EXPLAIN` startup costs of `812.86` and `408.77`.)

Step 5 — Estimated Startup Pages (`startupPages`): `startupPages = costs.numIndexPages * ratio`, giving **`202.4544` pages** (499 × 0.40572022 = 202.454390, or `202.4543` under 6-decimal `0.405720`) for FP32 and **`101.4301` pages** (250 × 0.40572022 = 101.430055, or `101.4300` under `0.405720`) for FP16 at N = 1,001.

Step 6 — Intentional TOAST-Compensation Guard (`startupPages > relpages && ratio < 0.5`): Finally, `src/hnsw.c` executes:

```
/* Adjust cost if needed since TOAST not included in
seq scan cost */
if (startupPages > path->indexinfo->rel->pages &&
ratio < 0.5)
{
    /* Change random page cost to sequential page
cost */
    costs.indexStartupCost -= startupPages *
(costs.spc_random_page_cost - spc_seq_page_cost);
    /* Remove cost of extra pages */
    costs.indexStartupCost -= (startupPages - path-
>indexinfo->rel->pages) * spc_seq_page_cost;
}
```

Combining both subtractions defines the TOAST-compensation startup discount Δ:

$$\Delta = \text{startupPages} \cdot (\text{random_page_cost} - \text{seq_page_cost}) + (\text{startupPages} - \text{relpages}) \cdot \text{seq_page_cost} = \text{startupPages} \cdot \text{random_page_cost} - \text{relpages} \cdot \text{seq_page_cost} \tag{5}$$

When the guard is **True**, subtracting Δ cancels startupPages · random_page_cost, leaving:

$$\text{indexStartupCost}_{\text{adjusted}} = \text{relpages} \cdot \text{seq_page_cost} + N \cdot (\text{cpu_index_tuple_cost} + \text{cpu_operator_cost}) \cdot \text{ratio} \tag{6}$$

At N = 1,001 ($\text{ratio}_{\text{cal}}$ = 0.40572022 < 0.5), `fp32_extended` (`relpages = 7 < 202.4544`) satisfies the guard (Δ = 607.3632 + 195.4544 = 802.8176 under $\text{ratio}_{\text{cal}}$, or `802.8172` under `0.405720`; **`802.82`** at two decimals), reducing `*indexStartupCost` to 7 × 1.0 + 3.045945 = **`10.0459`** (**`10.05`** in `EXPLAIN` across `P1, P4, P7`). For inline `fp32_plain` (`relpages = 501`) and `fp16_extended` (`relpages = 201`), `startupPages <= relpages` disables the workaround (Δ = 0.00), leaving the unadjusted generic HNSW startup estimates of **`812.86`** (`812.8635` calibrated / `812.8632` baseline, an **`80.91`× unrounded / `80.88`× display-rounded ratio** vs. `10.0459` / `10.05`) and **`408.77`** (`408.7662` calibrated / `408.7660` baseline, a **`40.69`× unrounded / `40.67`× display-rounded ratio**).

C. Three-Level Cost Propagation, Competing `Seq Scan` + `Sort`, and 1% Fuzzy Tie-Breaking

1. **Level 2 (`cost_index` and Mackert–Lohman `index_pages_fetched`):** In `costsize.c` [4], `cost_index` sets $\text{IndexScan}_{\text{startup}}$ = *indexStartupCost. For an uncorrelated index (`indexCorrelation = 0.0`), `cost_index` computes random heap page fetches via `index_pages_fetched(N, T, index->pages, root)` (implementing the Mackert and Lohman LRU buffer model [22]): when T = relpages ≤ b, pages_fetched = $\lceil \min(T, 2TN/(2T+N)) \rceil$, which evaluates to T = relpages whenever $2TN/(2T+N) > T - 1 \iff N > \mathbf{2T(T-1)/(T+1)}$. Across all 54 validated (`scale, representation`) storage configurations, `index_pages_fetched` evaluates identically to `relpages`. Thus:

$$\text{IndexScan}_{\text{run}} = (\text{indexTotalCost} - \text{IndexScan}_{\text{startup}}) + \text{relpages} \cdot \text{random_page_cost} + N \cdot (\text{cpu_tuple_cost} + \text{cpu_operator_cost}) \tag{7}$$

and $\text{IndexScan}_{\text{total}} = \text{IndexScan}_{\text{startup}} + \text{IndexScan}_{\text{run}}$ is invariant to Δ (`2044.02`, `4020.02`, and `1824.02` at N = 1,001, P4).

2. **Level 3 (`create_limit_path` for `LIMIT K`):** Scaling $\text{IndexScan}_{\text{run}}$ by K/N (K = 10) propagates $\Delta_{\text{Limit}} = (1 - K/N)\Delta$ (`794.80` units at N = 1,001, P4) into the root `Limit` total cost:

$$\text{Limit}_{\text{total, HNSW}} = \text{IndexScan}_{\text{startup}} + (K / N) \cdot \text{IndexScan}_{\text{run}} \tag{8}$$

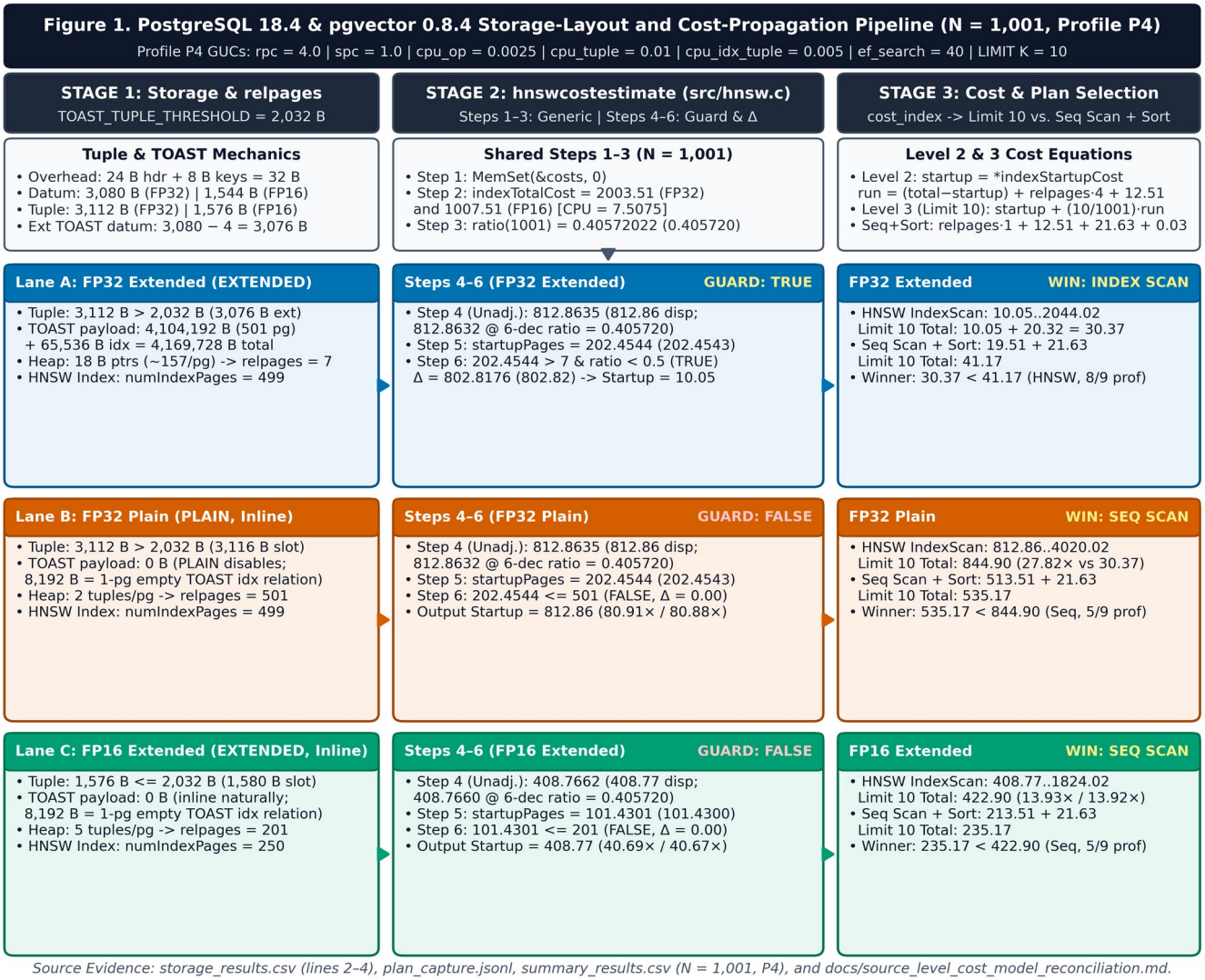
yielding Profile P4 root `Limit` total costs of **`30.37`** (`30.3654`) for `fp32_extended` versus **`844.90`** (`844.9027`, **`27.82`×**) for `fp32_plain` and **`422.90`** (`422.9044`, **`13.93`× unrounded / `13.92`× display**) for `fp16_extended`.

3. **Competing `Limit -> Sort -> Seq Scan` Path (`cost_seqscan + cost_tuplesort`):** Pricing main-heap sequential scan (relpages · seq_page_cost + N(cpu_tuple_cost + cpu_operator_cost)), bounded top-K heapsort comparisons (2.0 · cpu_operator_cost · N $\log_2(2.0\ K)$), and K output tuples (K · cpu_operator_cost) gives the exact closed-form equation:

$$\text{Limit}_{\text{total, Seq}} = \text{relpages} \cdot \text{seq_page_cost} + N \cdot (\text{cpu_tuple_cost} + \text{cpu_operator_cost}\,[1 + 2\log_2(2K)]) + K \cdot \text{cpu_operator_cost} \quad (9)$$

At N = 1,001 Profile P4, (9) yields **`41.17`** (`fp32_extended`), **`535.17`** (`fp32_plain`), and **`235.17`** (`fp16_extended`).

4. **Fuzzy Path Comparison (`compare_path_costs_fuzzily` in `pathnode.c`):** PostgreSQL compares candidate paths using `STD_FUZZ_FACTOR = 1.01` (1%) [4]. If two paths' total costs lie within 1% ($\max(C_1, C_2) \le 1.01\min(C_1, C_2)$), they are considered fuzzily tied on `total_cost` and PostgreSQL selects the path with the lower `startup_cost` (when $S_{\text{winner}} < S_{\text{loser}}$ / 1.01). Fig. 1 synthesizes this three-stage physical storage, `hnswcostestimate` guard evaluation, and `LIMIT 10` cost-propagation pipeline across all three vector configurations at N = 1,001 under canonical Profile P4, and Table I provides the complete step-by-step numerical reconstruction.



Source Evidence: storage_results.csv (lines 2–4), plan_capture.jsonl, summary_results.csv (N = 1,001, P4), and docs/source_level_cost_model_reconciliation.md.

Fig. 1. PostgreSQL 18.4 and `pgvector` 0.8.4 Storage-Layout and Cost-Propagation Pipeline at N = 1,001 Under Default Planner Profile P4 (`random_page_cost = 4.0`, `seq_page_cost = 1.0`, `cpu_operator_cost = 0.0025`, `cpu_tuple_cost = 0.01`, `cpu_index_tuple_cost = 0.005`, `hnsw.ef_search = 40`, `LIMIT 10`). Stage 1 contrasts physical heap and TOAST storage across the three vector configurations: **Lane A (`FP32 Extended, vector(768) with EXTENDED storage`)** externalizes `3,076 B` uncompressed TOAST datums (`toast_table_bytes = 4,104,192 B` across `501` TOAST chunk pages plus `toast_index_bytes = 65,536 B` across 8 TOAST B-tree index pages, totaling `toast_total_bytes = 4,169,728 B` [509 pages]), leaving only `18 B` `varatt_external` TOAST pointers in the main heap (~157 pointers/page, `relpages = 7`, `numIndexPages = 499`); **Lane B (`FP32 Plain, vector(768) with PLAIN storage`)** disables out-of-line TOAST (`toast_table_bytes = 0 B`, `0` TOAST chunk pages; PostgreSQL still allocates `toast_index_bytes = 8,192 B` [1 empty metapage] for the TOAST B-tree index relation, giving `toast_total_bytes = 8,192 B`), forcing `3,108 B` raw tuples (`3,112 B` `MAXALIGN` page footprint; 2 tuples/page, `relpages = 501`, `numIndexPages = 499`) inline; **Lane C (`FP16 Extended, halfvec(768) with EXTENDED storage`)** reduces raw tuple length to `1,572 B` (`1,576 B` `MAXALIGN` page footprint `<= 2,032 B` `TOAST_TUPLE_THRESHOLD`), keeping tuples inline naturally without out-of-line TOAST chunks (`toast_table_bytes = 0 B`, `toast_index_bytes = 8,192 B`, `toast_total_bytes = 8,192 B`, 5 tuples/page, `relpages = 201`, `numIndexPages = 250`). Stage 2 traces `hnswcostestimate` (`src/hnsw.c`, TOAST-compensation guard at lines 187–194): at N = 1,001, calibrated traversal ratio ratio(1,001) = 0.40572022 (`0.405720` in 6-decimal log export) yields `startupPages = 202.4544` (`202.4543` @ 6-dec) for `FP32` and `101.4301` (`101.4300` @ 6-dec) for `FP16`. The two-condition TOAST startup-cost discount Δ (`startupPages > path->indexinfo->rel->pages && ratio < 0.5`) fires **exclusively** for `FP32 Extended` (`202.4544 > 7`), subtracting Δ = 802.8176 (`802.82` display) to reduce `*indexStartupCost` from `812.86` (`812.8635`) to `10.05` (`10.0459`; `80.88×` display ratio / `80.91×` unrounded vs. `FP32 Plain`, and `40.67×` display / `40.69×` unrounded vs. `FP16 Extended` `408.77` [`408.7662`]). Stage 3 propagates `*indexStartupCost` into PostgreSQL's `cost_index` and top-K `LIMIT 10` interpolation, yielding unforced `Index Scan` selection for `FP32 Extended` (`30.37 < 41.17`, `8/9` profiles) but unforced `Seq Scan + Sort` selection for `FP32 Plain` (`535.17 < 844.90`, `5/9` profiles) and `FP16 Extended` (`235.17 < 422.90`, `5/9` profiles).

TABLE I. THREE-LEVEL NUMERICAL COST RECONSTRUCTION AT CANONICAL PROFILE P4 (N = 1,001, `RANDOM_PAGE_COST = 4.0`, `SEQ_PAGE_COST = 1.0`, `CPU_OPERATOR_COST = 0.0025`)

Cost Component / Planner Level	Exact Source Formula (`pgvector 0.8.4` & PG 18.4)	`FP32 Extended`	`FP32 Plain`	`FP16 Extended`
Catalog & Generic Estimate (`genericcostestimate`)				
Main-heap pages (`relpages`)	`path->indexinfo->rel->pages`	7	501	201
Index tuples (`costs.numIndexTuples`)	`index->tuples` (`indexSelectivity = 1.0`)	1001	1001	1001
Index pages (`costs.numIndexPages`)	`index->pages`	499	499	250
Level 1: Index AM (`hnswcostestimate`)				
Index total cost (`*indexTotalCost`)	`numIndexPages * 4.0 + 1001 * 0.0075`	**2003.51** (2003.5075)	**2003.51** (2003.5075)	**1007.51** (1007.5075)
HNSW traversal selectivity (`ratio`)	`(32 + 54.152441 * ln(1001)) / 1001` (calibrated)*	0.40572022 (0.405720)	0.40572022 (0.405720)	0.40572022 (0.405720)
Unadjusted generic HNSW startup cost	`(numIndexPages * 4.0) * ratio + 3.045945`	**812.86** (812.8635)*	**812.86** (812.8635)*	**408.77** (408.7662)*
Estimated startup pages (`startupPages`)	`costs.numIndexPages * ratio`	202.4544 (202.4543)	202.4544 (202.4543)	101.4301 (101.4300)
Guard `startupPages > relpages && ratio < 0.5`	TOAST-compensation guard in `src/hnsw.c`	**True** (`202.45 > 7`)	**False** (`202.45 <= 501`)	**False** (`101.43 <= 201`)
Adjustment term 1 / term 2	`startupPages * 3.0` / `(startupPages - relpages) * 1.0`	607.3632 / 195.4544	0.0000 / 0.0000	0.0000 / 0.0000
Total startup adjustment (Δ at P4)	startupPages· 4.0 - relpages· 1.0	**802.82** (802.8176)*	**0.00**	**0.00**
Output index startup cost (`*indexStartupCost`)	Initial - Δ (`80.91×` / `40.69×` unrounded; `80.88×` / `40.67×` display)	**10.05** (10.0459)	**812.86** (812.8635)*	**408.77** (408.7662)*
Level 2: `Index Scan` Path (`cost_index`)				
`Index Scan` startup / run cost	`*indexStartupCost` / `(*indexTotalCost - *indexStartupCost) + relpages*4.0 + 12.5125`	**10.05** / **2033.97**	**812.86** / **3207.16**	**408.77** / **1415.25**
`Index Scan` total cost (`EXPLAIN` capture)	`IndexScan_startup + IndexScan_run`	**2044.02**	**4020.02**	**1824.02**
Level 3: Root `Limit (rows=10)` over `Index Scan` (`forced_hnsw`)				
`Limit` startup / run cost	`IndexScan_startup` / `(10 / 1001) * IndexScan_run`	**10.05** / **20.32**	**812.86** / **32.04**	**408.77** / **14.14**
`Limit` total cost (`est_total_cost`, `forced_hnsw`)	`Limit_startup + Limit_run` (27.82× / 13.93× unrounded; 27.82× / 13.92× display)	**30.37** (30.3654)	**844.90** (844.9027)	**422.90** (422.9044)
Competing `Limit (rows=10)` over `Sort` + `Seq Scan` (`exact`)				
`Seq Scan` total / Bounded `Sort` comparisons	`relpages * 1.0 + 12.5125` / `2 * 0.0025 * 1001 * log2(20)`	**19.51** / 21.6313	**513.51** / 21.6313	**213.51** / 21.6313
`Limit` startup / total cost (`exact`)	`Sort_startup` / `+ 10 * 0.0025`	**41.14** / **41.17**	**535.14** / **535.17**	**235.14** / **235.17**
Actual Unforced `planner_selected` Path at P4	Captured in unforced `EXPLAIN`	**`Index Scan` (30.37)**	**`Seq Scan` (535.17)**	**`Seq Scan` (235.17)**

Note: Using the 8-decimal calibrated ratio $ratio_{cal}$ = 0.40572022 (`0.4057202163` full precision in `[Supplementary Material – Repository/DOI Placeholder]`), the exact 4-decimal unadjusted startup costs are 1996.0 × 0.40572022 + 3.045945 = 809.8176 + 3.0459 = 812.8635 (FP32) and 1000.0 × 0.40572022 + 3.045945 = 405.7202 + 3.0459 = 408.7662 (FP16), with `startupPages` = 202.4544 and 101.4301 and Δ = 607.3632 + 195.4544 = 802.8176. Using the 6-decimal truncated `ratio = 0.405720` with 4-decimal rounded `startupPages = 202.4543` and CPU term `3.0460` gives 812.8632, 408.7660, and Δ = 607.3629 + 195.4543 = 802.8172. Under both parameterizations, the captured two-decimal `EXPLAIN` display values (812.86, 408.77, 802.82, 10.05, 30.37, 844.90, 422.90), the unrounded ratios (812.8635 / 10.0459 = 80.9146× ≈ **80.91× and 408.7662 / 10.0459 = 40.6897× ≈ **40.69×** for startup; 844.9027 / 30.3654 = **27.82×** and 422.9044 / 30.3654 = **13.93×** for root `Limit` total), and the two-decimal `EXPLAIN` display ratios (80.88×, 40.67×, 27.82×, 13.92×) are identical.*

III. RELATED WORK

1. **PostgreSQL Vector Indexing and Practitioner TOAST Observations:** Yang et al. [6] introduced PASE for IVFFlat and HNSW indexing inside PostgreSQL, and `pgvector` [2], [3] implements exact vector scans, IVFFlat, and HNSW [5] with support for 16-bit `halfvec` attributes. Zhang, Liu, and Wang [7] compared PostgreSQL/PASE against standalone Faiss, identifying seven execution-engine root causes (`RC#1`–`RC#7`), including missing SGEMM in IVF build (`RC#1`), buffer-pool page indirection and tuple access overhead (`RC#2`), global top-K heap locking (`RC#3`), and 8 KiB page-layout space amplification (`RC#4`). In a non-archival practitioner conference presentation at PGConf.EU 2025, Otochkin [21] noted that FP32 vectors exceeding ~2 KiB (`>500` dimensions) move to `pg_toast`, resist compression, and incur 3–4 buffer accesses per row during sequential KNN scans unless `STORAGE PLAIN` is used. Complementary to Zhang et al.'s [7] execution-engine study and Otochkin's [21] practitioner talk, we reconstruct `pgvector` 0.8.4's `hnswcostestimate` [3] and characterize how inline `halfvec(768)` and `STORAGE PLAIN` layouts interact with `startupPages > relpages && ratio < 0.5` across cost profiles and scales.

2. **Hybrid Vector-Relational Processing and Query Optimization:** Wei et al. [8] developed AnalyticDB-V to fuse structured predicates and vector search via selectivity-guided plan selection, and Zhang et al. [9] proposed VBASE to unify

online vector similarity and relational queries via relaxed monotonicity. In classical relational optimization, Mackert and Lohman [22] validated the LRU buffer I/O model (`index_pages_fetched`), and Leis et al. [24] demonstrated that cardinality-estimation errors typically outweigh cost-model errors in multi-join plans. Extending cardinality estimation to vector-augmented analytical queries (VAQs) with multi-table joins and range predicates, Kim et al. [10] introduced Exqutor, combining Exact Cardinality Query Optimization (ECQO, via planning-time ANN range probing with result reuse) and Q-error-guided adaptive sampling. Whereas Leis et al. [24] and Exqutor [10] focus on join and range-predicate cardinality estimation, our study examines single-table top-K ordering queries (`LIMIT 10`) where output cardinality is fixed (K = 10) and plan selection is governed jointly by relation scale (N), the calibrated HNSW entry-traversal ratio (ratio(N) < 0.5), physical storage layout (`relpages` under inline versus TOASTed storage), and planner cost settings (`random_page_cost`, `cpu_operator_cost`) inside `hnswcostestimate` and `cost_seqscan`.

3. **SSD-Aware HNSW I/O, Reduced Precision, and Tie-Aware Retrieval:** Shim et al. [11] optimized physical NVMe SSD I/O during forced `pgvector` HNSW searches via `io_uring` batching and colocation, whereas we analyze PostgreSQL 18.4 shared-buffer traffic and `BAS_BULKREAD` ring-buffer behavior across exact and unforced scans. Unlike codebook-based Product Quantization (PQ) [12], `pgvector`'s `halfvec(768)` [2] applies a direct scalar conversion to IEEE-754 `binary16` format [25], halving HNSW index size while keeping tuples below `TOAST_TUPLE_THRESHOLD`. For retrieval evaluation, we distinguish McSherry and Najork's [13] expected IR metrics over random permutations of exact score ties from Aumüller et al.'s [23] ϵ-threshold ANN recall, treating exact duplicate vectors as an equivalence class and genuine ϵ-near-ties as a secondary tolerance bound (Appendix A).

IV. Experimental Methodology

A. Dataset, Representations, and Verified Query `q0`–`q19` Provenance

We use a prepared passage collection from the `BeIR/msmarco` corpus split [14], [15] (`5,457` source documents segmented into `10,001` chunks of `300` characters with `50`-character overlap, source TSV SHA-256 `cdca54aa...34e8b`; the upstream filter selecting those `5,457` documents is not separately recorded [Supplementary Material – Repository/DOI Placeholder]). Three primary benchmark scales ($N \in$ {1,001, 5,000, 10,000}) and sixteen intermediate sweep scales ($N \in$ {500, 700, 750, 780, 785, 786, 787, 788, 790, 850, 1,250, 1,500, 1,750, 2,000, 3,000, 4,000}) are constructed as deterministic nested prefix slices (`ORDER BY chunk_id ASC LIMIT N`); because each scale N is a prefix slice rather than an independent random sample, smaller scales are strict subsets of larger scales and corpus composition evolves monotonically with N. Passages and queries are represented as 768-dimensional embeddings using the mutable Ollama model tag `nomic-embed-text:latest` [16]. The exact model artifact digest and Ollama runtime build were not recorded; consequently, the original embedding-generation environment cannot be reconstructed exactly from the available provenance. At each scale N, three two-column tables (`chunks_{scale}_{rep}`) are provisioned with 100% `chunk_id` parity, explicit `ALTER COLUMN embedding SET STORAGE` policy assignment ([Supplementary Material – Repository/DOI Placeholder]), a single HNSW index build (m = 16, $ef_{construction}$ = 64), and immediate `ANALYZE`: (1) `fp32_extended` (`vector(768)`, `SET STORAGE EXTENDED`, `3,076 B` uncompressed external `pg_column_size`), (2) `fp32_plain` (`vector(768)`, `SET STORAGE PLAIN`, `3,080 B` inline datum), and (3) `fp16_extended` (`halfvec(768)`, `SET STORAGE EXTENDED`, `1,544 B` inline datum).

The workload comprises 20 fixed topical queries (`q0`–`q19`) pinned in the benchmark harness ([Supplementary Material – Repository/DOI Placeholder], commit `3f2beee66cecd67aa8fc4cec69c364b92c07b884`). In particular, **canonical query `q0` (`query_id = 0`) is "`what was the purpose of the manhattan project`"** (retrieving the exact FP32 reference top-10 `chunk_id` sequence `[11, 4, 5, 14, 3, 10, 12, 9, 1, 7]` in rank order `1..10` across N = 1,001, 5,000, and 10,000), whose precomputed 768-float embedding is verified by the UTF-8 JSON SHA-256 hash `4a0428709013d77d8d284b369cb144734421bfdc4f6e4a83faf17db8114cf478` ([Supplementary Material – Repository/DOI Placeholder]). Likewise, `q8` (`query_id = 8`) is *"how do vector databases store and index embeddings"*, `q9` (`query_id = 9`) is *"difference between supervised and unsupervised machine learning"*, and `q17` (`query_id = 17`) is *"how do commercial airplane wings produce aerodynamic lift"*. All `fp16_extended` queries cast the query operand to `halfvec(768)` (`SELECT chunk_id, (embedding <=> %s::halfvec(768)) AS distance ... ORDER BY distance ASC LIMIT 10`), evaluating end-to-end 16-bit representation across both stored vectors and query operands.

B. Hardware, PostgreSQL 18.4 Configuration, and Planner Profiles (`P1`–`P9`)

Experiments ran on a Windows 11 (`10.0.26200`, `x86-64`) host with an Intel Core Ultra 9 185H CPU (16 physical cores, 22 logical processors), 32 GB RAM (`15.08 GiB` free at snapshot), local NVMe storage, PostgreSQL `18.4`, `pgvector 0.8.4`, Python `3.13.14`, and `psycopg 3.3.4`. Verified PostgreSQL 18.4 server parameters ([Supplementary Material – Repository/DOI Placeholder]) are `shared_buffers = 128MB` (`16,384` 8 KiB blocks), `effective_cache_size = 4GB`, `work_mem = 4MB`, `effective_io_concurrency = 16` (default), `io_combine_limit = 128kB` (16 blocks, default), and server-default `max_parallel_workers_per_gather = 2`. Scale N = 1,001 resides in `compression_aware_rag_exp`, and scales $N \in$ {5,000, 10,000} plus the 16 intermediate sweep scales reside in `compression_aware_rag_scale` (`32/32` pre-existing catalog objects verified unchanged).

We evaluate a 3 × 3 grid of `random_page_cost` ∈ {1.0, 4.0, 8.0} and `cpu_operator_cost` ∈ {0.0025, 0.0100, 0.0500} (`P1`–`P3` at `rpc = 1.0`; `P4`–`P6` at `rpc = 4.0`; `P7`–`P9` at `rpc = 8.0`), holding `seq_page_cost = 1.0`, `cpu_tuple_cost = 0.01`, `cpu_index_tuple_cost = 0.005`, `hnsw.ef_search = 40`, and `LIMIT = 10` fixed. Importantly, the benchmark harness sets `SET LOCAL max_parallel_workers_per_gather = 0` in every transaction to isolate single-worker access-path selection; thus, **Profile `P4` (`rpc = 4.0, cpu_op = 0.0025`) uses PostgreSQL's baseline page and CPU cost parameters under single-worker execution**, rather than the full out-of-the-box parallel server configuration, and extreme settings (`rpc = 8.0`, `cpu_op = 0.05`) serve as controlled sensitivity probes of the optimizer's cost equations rather than typical deployment recommendations. Three execution modes are tested via `SET LOCAL`: `planner_selected` (unforced), `exact` (`enable_indexscan = off`, `enable_bitmapscan = off`), and `forced_hnsw` (`enable_seqscan = off`).

C. Three-Stage Evaluation Protocol, 486-Cell Candidate-Cost Validation, and Timing Semantics

All queries execute inside read-only transactions with prepared-statement caching disabled (`prepare_threshold=None`, `prepare=False`):

1. **Deterministic Plan-Capture Stage (`4,860` plans across `81` unforced settings) and 18-Scale Sweep (`972` plans + `486` exact-validation plans across `486` settings):** In the 3-scale baseline (3 × 3 × 3 × 9 × 20 = 4,860 `EXPLAIN` captures), **0 of 243 cells** vary in plan or cost across `q0`–`q19`. Thus, the **distinct `(scale, representation, profile)` setting (27 per scale; `486` across the 18-scale sweep)** is the primary unit for planner decisions, and deterministic plan counts (`972` or `486`) are never treated as independent sample observations in statistical inference. In the 18-scale sweep (`[Supplementary Material – Repository/DOI Placeholder]`, using `q0` SHA-256 `4a042870...`), component timers recorded `5.750 s` (provisioning 48 tables), `31.670 s` (48 HNSW builds + `ANALYZE`), and `5.057 s` (`972` `EXPLAIN` captures in `planner_selected` and `forced_hnsw` modes); their sum (`42.477 s`) is an arithmetic sum of discrete stage timers, whereas continuous end-to-end wall-clock runtime is marked `unavailable` (total multi-stage workflow span `622.086 s`, finalization pass `2.086 s`). In our 486-cell exact-mode cost-validation pass (`[Supplementary Material – Repository/DOI Placeholder]`), read-only `EXPLAIN` was additionally executed under forced `exact` mode across **all `486/486` cells** (18 × 3 × 9), confirming that our closed-form `Seq Scan + Top-k Sort` equation (9) matches PostgreSQL 18.4's displayed startup and total costs in **`486/486` cells (`100.0%`) with `0.0000` maximum cost difference (identical at two-decimal `EXPLAIN` precision)**, validating the estimated cost model across all 486 cells without claiming physical latency measurements for the 16 un-timed intermediate scales.

2. **Client Round-Trip Timing Stage (`9,900` trials) vs. Buffer-Diagnostic Stage (`540` `EXPLAIN ANALYZE` runs):** The client timing stage measures `planner_selected` across `P1`–`P9` (`8,100` trials) and `exact` and `forced_hnsw` at `P4` (`900` trials each) across 5 randomized passes per scale (preceded by 2 warm-up passes) within single-session windows (`12.07 s` at 1,001, `16.36 s` at 5,000, `32.47 s` at 10,000). Because `time.perf_counter()` brackets `cur.execute(..., prepare=False)` and `cur.fetchall()`, this stage captures **end-to-end client round-trip latency** (including driver serialization, Windows loopback IPC, server query planning, execution, and result transfer). The separate diagnostic stage runs `EXPLAIN (ANALYZE, BUFFERS, SETTINGS, FORMAT JSON)` at Profile P4 (`540` runs in `[Supplementary Material – Repository/DOI Placeholder]`) to record root-node 8 KiB shared-buffer block counts. Because `TIMING OFF` was not specified during the diagnostic pass, its `execution_time_ms` includes per-node `EXPLAIN ANALYZE` instrumentation overhead from a separate 1-pass run (in `1,001` `fp32_extended` `exact`, diagnostic median execution time is `10.489 ms` vs. client round-trip median `5.127 ms`); therefore, diagnostic timings are reported strictly as **instrumented diagnostic measurements**, not as pure server wall-clock speedup ratios. Across the 9 Profile P4 `forced_hnsw` cells, the inferred difference between client median latency and diagnostic server (`Planning + Execution`) median time spans **`1.11–1.64 ms`** (`1.19–1.68 ms` relative to `Execution Time` alone), rather than a fixed constant.

3. **Statistical Inference (N = 20 query blocks):** Averaging the 5 passes into N = 20 query-level means, we evaluate (i) tie-corrected Friedman omnibus tests (χ^2_8, BH FDR-adjusted [17], [18]) across `P1`–`P9` (which act as within-session negative controls at $N \geq 5{,}000$ where all 9 profiles execute the identical HNSW plan), (ii) paired two-sided Wilcoxon signed-rank tests [17] with Holm adjustment [19] and rank-biserial r_{rb} [20] across representations under P4 `forced_hnsw`, and (iii) P4 `exact` vs. `forced_hnsw` paired Wilcoxon tests (BH-adjusted) alongside 30,000-resample query-cluster bootstrap 95% CIs. We report both **continuity-corrected (`+0.5`) asymptotic normal Wilcoxon *p*-values** ($z = (|W - \mu_W| - 0.5)/\sigma_W$) and **exact combinatorial Wilcoxon *p*-values** (2^{20} sign assignments) consistently across all tables and text. For primary conventional `Recall@10` (n = 20 query-level observations per `exact` and `forced_hnsw` cell, as well as the n = 19 sensitivity subset excluding `Q9`), we compute percentile 95% confidence intervals (`2.5th` and `97.5th` order-statistic percentiles at indices `250` and `9,750`) by resampling the sorted query-level observations (`query_id 0..19`) with replacement across `10,000` bootstrap replications using an independent `random.Random(42)` stream (`rng.choice`) per series (`[Supplementary Material – Repository/DOI Placeholder]`). These intervals characterize finite-sample variability over our fixed topical 20-query workload (`q0`–`q19`) and do not establish generalization to an arbitrary universe of semantic queries (degenerating to `[1.0000, 1.0000]` in `exact` cells where all 20 queries achieve `1.0000`).

V. Empirical Results

A. Physical Storage Footprint and Shared-Buffer Dynamics (RQ2)

Table II summarizes physical catalog storage and Profile P4 root-plan-node 8 KiB shared-buffer block counts (`[Supplementary Material – Repository/DOI Placeholder]`).

TABLE II. PHYSICAL STORAGE FOOTPRINT AND CANONICAL PROFILE P4 ROOT-NODE SHARED-BUFFER BLOCK ACCESS

Scale (*N*)	Representation	Policy	Datum (B)	relpages	Heap (Bytes)	TOAST Total (Bytes)	HNSW Index (Bytes)	Total Inclusive (Bytes)	Total Red. vs FP32 Ext	Exact Scan Shared-Buffer Blocks (Hit / Read; Total)*	Forced HNSW Shared-Buffer Blocks (Hit / Read; Total)*	Planner-Selected Shared-Buffer Blocks (Hit / Read; Total)*
1,001	FP32 Extended	EXTENDED	3,076	7	57,344	4,169,728	4,087,808	8,413,184	Baseline (0.00%)	3010.0 / 0.00 (3010.0)	508.5 / 0.00 (508.5)	508.5 / 0.00 (508.5)
1,001	FP32 Plain	PLAIN	3,080	501	4,104,192	8,192	4,087,808	8,265,728	+1.75%	501.0 / 0.00 (501.0)	484.95 / 0.00 (484.95)	501.0 / 0.00 (501.0)
1,001	FP16 Extended	EXTENDED	1,544	201	1,646,592	8,192	2,048,000	3,768,320	**+55.21%**	201.0 / 0.00 (201.0)	486.55 / 0.00 (486.55)	201.0 / 0.00 (201.0)
5,000	FP32 Extended	EXTENDED	3,076	32	262,144	20,725,760	20,422,656	41,607,168	Baseline (0.00%)	15032.0 / 0.00 (15032.0)	696.95 / 0.00 (696.95)	696.95 / 0.00 (696.95)
5,000	FP32 Plain	PLAIN	3,080	2,500	20,480,000	8,192	20,430,848	41,082,880	+1.26%	2500.0 / 0.00 (2500.0)	684.15 / 0.00 (684.15)	684.15 / 0.00 (684.15)
5,000	FP16 Extended	EXTENDED	1,544	1,000	8,192,000	8,192	10,215,424	18,579,456	**+55.35%**	1000.0 / 0.00 (1000.0)	671.25 / 0.00 (671.25)	671.25 / 0.00 (671.25)
10,000	FP32 Extended	EXTENDED	3,076	64	524,288	41,426,944	40,779,776	83,050,496	Baseline (0.00%)	30064.0 / 0.00 (30064.0)	715.5 / 29.35 (744.85)	744.85 / 0.00 (744.85)
10,000	FP32 Plain	PLAIN	3,080	5,000	40,960,000	8,192	40,779,776	82,034,688	+1.22%	1117.0 / 3883.00 (5000.0)	642.95 / 88.05 (731.0)	731.0 / 0.00 (731.0)
10,000	FP16 Extended	EXTENDED	1,544	2,000	16,384,000	8,192	20,398,080	37,068,800	**+55.37%**	1906.55 / 93.45 (2000.0)	679.5 / 36.40 (715.9)	715.9 / 0.00 (715.9)

**Note: In Table II ([Supplementary Material – Repository/DOI Placeholder]), TOAST Total (Bytes) reports pg_relation_size(reltoastrelid) + pg_indexes_size(reltoastrelid) = toast_table_bytes + toast_index_bytes. For inline FP32 Plain and FP16 Extended, the external TOAST chunk heap is empty (toast_table_bytes = 0 B, 0 chunk pages) while the associated TOAST B-tree index allocates a single empty 8 KiB metapage (toast_index_bytes = 8,192 B), yielding toast_total_bytes = 8,192 B at all scales. For out-of-line FP32 Extended, toast_table_bytes + toast_index_bytes = toast_total_bytes equals 4,104,192 B (501 chunk pages) + 65,536 B (8 index pages) = 4,169,728 B (509 pages) at N = 1,001; 20,480,000 B (2,500 pages) + 245,760 B (30 pages) = 20,725,760 B (2,530 pages) at N = 5,000; and 40,960,000 B (5,000 pages) + 466,944 B (57 pages) = 41,426,944 B (5,057 pages) at N = 10,000. Total Inclusive (Bytes) is measured using PostgreSQL's pg_total_relation_size(relid) and equals Heap (Bytes) + TOAST Total (Bytes) + all_indexes_bytes (where all_indexes_bytes = pk_index_bytes + hnsw_index_bytes, including both the B-tree primary-key index on chunk_id [40,960 B at N = 1,001, 131,072 B at N = 5,000, and 245,760 B at N = 10,000] and the separately displayed HNSW Index (Bytes)) plus an unallocated auxiliary-fork residual (reflecting non-main relation forks such as FSM and VM included by pg_total_relation_size(relid) but not separately queried by fork in the storage capture) of 57,344 B (N = 1,001), 65,536 B (N = 5,000), and 73,728 B (N = 10,000) for out-of-line FP32 Extended; 24,576 B, 32,768 B, and 40,960 B for inline FP32 Plain; and 24,576 B, 32,768 B, and 32,768 B for inline FP16 Extended. The displayed HNSW Index (Bytes) column is therefore only one component of Total Inclusive (Bytes), and the three visible byte columns (Heap + TOAST Total + HNSW Index) do not alone sum to Total Inclusive (Bytes) without adding pk_index_bytes and the auxiliary-fork residual. Shared-buffer counts are in 8 KiB blocks (Shared Hit Blocks / Shared Read Blocks [Mean Total]).*

1. **Storage Savings and TOAST Inversion:** `fp16_extended` reduces total inclusive bytes by **55.21%**, **55.35%**, and **55.37%** and HNSW index bytes by **49.90%**, **49.98%**, and **49.98%** at N = 1,001, 5,000, and 10,000. Under `fp32_extended` at N = 10,000, `pg_toast` holds `41,426,944 B` versus `524,288 B` (64 blocks) in the main heap (**98.75%** of table storage).

2. **Aggregate `relpages + 3N` TOAST Buffer Access:** In `exact` (`Seq Scan`) mode, inline tables access `relpages` total blocks per query (`501.0`, `2,500.0`, `5,000.0` for `fp32_plain`; `201.0`, `1,000.0`, `2,000.0` for `fp16_extended`), whereas TOASTed `fp32_extended` accesses an exact aggregate root-counter total of **`relpages + 3N` blocks** (`3,010.0`, `15,032.0`, and `30,064.0` blocks with zero variance across queries, reflecting per-row TOAST B-tree index and chunk lookups in `toast_fetch_datum`).

3. **Qualification of `BAS_BULKREAD` Correlation at N = 10,000:** In PostgreSQL 18.4 (`src/backend/access/heap/heapam.c`, `initscan`), sequential scans allocate a `BAS_BULKREAD` buffer strategy when `scan->rs_nblocks > NBuffers / 4` [4]. With `shared_buffers = 128MB` (`NBuffers = 16,384` blocks), the threshold is **`4,096` blocks**. At N = 10,000, `fp32_plain` (`relpages = 5,000 > 4,096`) crosses this threshold (`3,883.00 Shared Read Blocks` / `1,117.0` hits), whereas `fp16_extended` (`relpages = 2,000 <= 4,096`, `93.45` reads / `1,906.55` hits) and `fp32_extended` (`relpages = 64 <= 4,096`, `0.00` reads) do not. In PostgreSQL 18, bulk-read ring buffer behavior interacts dynamically with streaming read settings (`effective_io_concurrency = 16`, `io_combine_limit = 128kB` = 16 blocks) rather than enforcing a static 32-block ring under all conditions. Moreover, because our experiment did not isolate buffer-pool recycling from CPU distance and sorting work, the `3,883.00` vs. `93.45` shared-read contrast is reported strictly as a **buffer-strategy and working-set correlation**, not a proven sole causal explanation of `exact`-scan latency differences.

B. 18-Scale Planner Transition Characterization and Crossover Intervals (RQ1)

Across the three primary scales ($N \in$ {1,001, 5,000, 10,000}, 27 distinct settings per scale), the unforced planner selects `Seq Scan` in **`11/27` (`40.7%`) settings at N = 1,001** (`1/9` on `fp32_extended` [`P7`], `5/9` on `fp32_plain` [`P4, P5, P7, P8, P9`], and `5/9` on `fp16_extended` [`P4, P5, P7, P8, P9`]) and **`16/27` (`59.3%`) HNSW settings**, before converging to **`27/27` (`100.0%`) HNSW settings at N = 5,000 and 10,000**.

Tables II-B and II-C report the complete 18-scale plan-only sweep ($N \in [500, 5{,}000]$, 486 settings: **280/486 [57.6%] HNSW Index Scan** and **206/486 [42.4%] Seq Scan**, with all 486/486 competing Seq Scan + Sort startup and total costs verified to 0.0000 difference—identical at two-decimal EXPLAIN precision—in [Supplementary Material – Repository/DOI Placeholder]).

TABLE II-B. 18-SCALE SWEEP: CATALOG PAGE COUNTS, CALIBRATED TRAVERSAL RATIO, GUARD ACTIVATION (STARTUPPAGES > RELPAGES && RATIO < 0.5), AND UNFORCED HNSW SETTINGS

Scale (N)	ratio(N)	ratio < 0.5	FP32 Extended (relpages / idx_pages / startupPages / Guard / HNSW Profiles)	FP32 Plain (relpages / idx_pages / startupPages / Guard / HNSW Profiles)	FP16 Extended (relpages / idx_pages / startupPages / Guard / HNSW Profiles)	Total HNSW Settings (/ 27)
500	0.737072	False	4 / 251 / 185.01 / **False** / 1/9 (P3)	250 / 251 / 185.01 / **False** / 3/9 (P1–P3)	100 / 126 / 92.87 / **False** / 3/9 (P1–P3)	**7 / 27** (25.9%)
700	0.552510	False	5 / 349 / 192.83 / **False** / 1/9 (P3)	350 / 349 / 192.83 / **False** / 3/9 (P1–P3)	140 / 175 / 96.69 / **False** / 4/9 (+P6)	**8 / 27** (29.6%)
750	0.520657	False	5 / 374 / 194.73 / **False** / 1/9 (P3)	375 / 374 / 194.73 / **False** / 3/9 (P1–P3)	150 / 188 / 97.88 / **False** / 4/9 (P1–P3, P6)	**8 / 27** (29.6%)
780	0.503355	False	5 / 389 / 195.81 / **False** / 1/9 (P3)	390 / 389 / 195.81 / **False** / 3/9 (P1–P3)	156 / 195 / 98.15 / **False** / 4/9 (P1–P3, P6)	**8 / 27** (29.6%)
785	0.500590	False	5 / 392 / 196.23 / **False** / 1/9 (P3)	393 / 392 / 196.23 / **False** / 3/9 (P1–P3)	157 / 197 / 98.62 / **False** / 4/9 (P1–P3, P6)	**8 / 27** (29.6%)
786	**0.500041**	**False**	6 / 392 / 196.02 / **False** / **1/9 (P3)**	393 / 392 / 196.02 / **False** / 3/9 (P1–P3)	158 / 197 / 98.51 / **False** / 4/9 (P1–P3, P6)	**8 / 27** (29.6%)
787	**0.499493**	**True**	6 / 393 / 196.30 / **True** / **8/9 (P1–P6, P8, P9)**	394 / 393 / 196.30 / **False** / 3/9 (P1–P3)	158 / 197 / 98.40 / **False** / 4/9 (P1–P3, P6)	**15 / 27** (55.6%)
788	0.498946	True	6 / 393 / 196.09 / **True** / 8/9 (not P7)	394 / 393 / 196.09 / **False** / 3/9 (P1–P3)	158 / 197 / 98.29 / **False** / 4/9 (P1–P3, P6)	**15 / 27** (55.6%)
790	0.497857	True	6 / 394 / 196.16 / **True** / 8/9 (not P7)	395 / 394 / 196.16 / **False** / 3/9 (P1–P3)	158 / 198 / 98.58 / **False** / 4/9 (P1–P3, P6)	**15 / 27** (55.6%)
850	0.467378	True	6 / 424 / 198.17 / **True** / 8/9 (not P7)	425 / 424 / 198.17 / **False** / 4/9 (+P6*)	170 / 213 / 99.55 / **False** / 4/9 (P1–P3, P6)	**16 / 27** (59.3%)
1,001	0.405720	True	7 / 499 / 202.45 / **True** / 8/9 (not P7)	501 / 499 / 202.45 / **False** / 4/9 (P1–P3, P6)	201 / 250 / 101.43 / **False** / 4/9 (P1–P3, P6)	**16 / 27** (59.3%)
1,250	0.334524	True	8 / 623 / 208.41 / **True** / 8/9 (not P7)	625 / 623 / 208.41 / **False** / 4/9 (P1–P3, P6)	250 / 312 / 104.37 / **False** / 4/9 (P1–P3, P6)	**16 / 27** (59.3%)
1,500	0.285352	True	10 / 748 / 213.44 / **True** / **9/9 (+P7)**	750 / 748 / 213.44 / **False** / 5/9 (+P5)	300 / 375 / 107.01 / **False** / 6/9 (+P5, P9)	**20 / 27** (74.1%)
1,750	0.249358	True	12 / 873 / 217.69 / **True** / 9/9 (All)	875 / 873 / 217.69 / **False** / 6/9 (+P4)	350 / 437 / 108.97 / **False** / 6/9 (P1–P3, P5, P6, P9)	**21 / 27** (77.8%)
2,000	0.221804	True	13 / 998 / 221.36 / **True** / 9/9 (All)	1000 / 998 / 221.36 / **False** / 7/9 (+P9)	400 / 500 / 110.90 / **False** / 7/9 (+P4)	**23 / 27** (85.2%)
3,000	0.155188	True	20 / 1498 / 232.47 / **True** / 9/9 (All)	1500 / 1498 / 232.47 / **False** / 7/9 (not P7, P8)	600 / 750 / 116.39 / **False** / 7/9 (not P7, P8)	**23 / 27** (85.2%)
4,000	0.120286	True	26 / 1995 / 239.97 / **True** / 9/9 (All)	2000 / 1995 / 239.97 / **False** / **9/9 (+P7, P8)**	800 / 998 / 120.05 / **False** / 8/9 (+P8)	**26 / 27** (96.3%)
5,000	0.101845	True	32 / 2493 / 253.90 / **True** / 9/9 (All)	2500 / 2494 / 254.00 / **False** / 9/9 (All)	1000 / 1247 / 127.00 / **False** / **9/9 (+P7)**	**27 / 27** (100.0%)

**At N = 850, fp32_plain Profile P6 selects Index Scan via PostgreSQL's STD_FUZZ_FACTOR = 1.01 (1%) fuzzy total-cost tie-break: HNSW root Limit total cost (846.04) and Seq Scan + Sort total cost (843.86) lie within the 1% fuzz factor (846.04 <= 1.01 * 843.86 = 852.30), after which the startup-cost comparison allows HNSW (814.52) to win over raw Seq Scan + Sort startup cost (843.36) because 814.52 < 843.36 / 1.01 = 835.01 (the 1% fuzzy startup threshold).*

TABLE II-C. OBSERVED PLANNER CROSSOVER INTERVALS [N_{SEQ}, N_{HNSW}] ACROSS ALL 27 (REPRESENTATION, PROFILE) TRAJECTORIES

Representation	Profile	(rpc, cpu_op)	Seq Scan Scales (/18)	HNSW Scales (/18)	Observed Crossover Interval [N_{Seq}, N_{HNSW}]	HNSW vs. Seq Scan Root Limit Total Cost Across Boundary	Controlling Optimizer Mechanism
FP32 Extended	P3	(1.0, 0.05)	0/18	18/18	Always HNSW at tested lower bound N=500*	@500: HNSW 207.42 < Seq 250.60	High cpu_op makes Seq Scan + Sort costlier even without Δ
FP32 Extended	P1, P2	(1.0, 0.0025–0.01)	6/18	12/18	**[786, 787]** (width 1)	P1 @786: 201.70 > 32.84 → @787: 14.10 < 32.87	ratio < 0.5 activates Δ at N = 787 (0.499493 < 0.5)
FP32 Extended	P4, P5, P6	(4.0, 0.0025–0.05)	6/18	12/18	**[786, 787]** (width 1)	P4 @786: 797.45 > 32.84 → @787: 29.31 < 32.87	ratio < 0.5 activates Δ (88.06× display / 88.08× unrounded P4 startup drop)
FP32 Extended	P8, P9	(8.0, 0.01–0.05)	6/18	12/18	**[786, 787]** (width 1)	P8 @786: 1594.86 > 89.76 → @787: 52.65 < 89.87	ratio < 0.5 activates Δ at N = 787
FP32 Extended	P7	(8.0, 0.0025)	12/18	6/18	**[1250, 1500]** (width 250)	@1250: 51.63 > 50.66 (relpages=8) → @1500: 53.75 < 61.19 (relpages=10)	Discrete relpages step (8 → 10) + linear Seq Scan growth
FP32 Plain	P1, P2, P3	(1.0, 0.0025–0.05)	0/18	18/18	Always HNSW at tested lower bound N=500*	P1 @500: HNSW 194.23 < Seq 267.08	At rpc = 1.0, startupPages (185.01) < inline relpages (250)
FP32 Plain	P6	(4.0, 0.05)	9/18	9/18	**[790, 850]** (width 60)	@790: 837.15 > 784.33 → @850: 846.04 vs. 843.86 (+0.258%)	STD_FUZZ_FACTOR = 1.01 total tie-break (HNSW startup 814.52 < raw Seq+Sort startup 843.36; < 843.36 / 1.01 = 835.01 threshold)
FP32 Plain	P5	(4.0, 0.01)	12/18	6/18	**[1250, 1500]** (width 250)	@1250: 873.47 > 758.15 → @1500: 894.76 < 909.76	Linear inline Seq Scan + Sort cost overtakes unadjusted HNSW
FP32 Plain	P4	(4.0, 0.0025)	13/18	5/18	**[1500, 1750]** (width 250)	@1500: 891.42 > 801.19 → @1750: 909.19 < 934.72	Linear inline Seq Scan + Sort cost overtakes unadjusted HNSW
FP32 Plain	P9	(8.0, 0.05)	14/18	4/18	**[1750, 2000]** (width 250)	@1750: 1836.49 > 1736.84 → @2000: 1867.37 < 1984.89	High cpu_op + inline relpages growth overtakes rpc = 8.0 HNSW
FP32 Plain	P7, P8	(8.0, 0.0025–0.01)	16/18	2/18	**[3000, 4000]** (width 1000)	P7 @3000: 1937.20 > 1602.35 → @4000: 1998.66 < 2136.46	Linear inline relpages (1,500 -> 2,000) overtakes rpc = 8.0 HNSW
FP16 Extended	P1, P2, P3	(1.0, 0.0025–0.05)	0/18	18/18	Always HNSW at tested lower bound N=500*	P1 @500: HNSW 98.44 < Seq 117.08	At rpc = 1.0, startupPages (92.87) < inline relpages (100)
FP16 Extended	P6	(4.0, 0.05)	1/18	17/18	**[500, 700]** (width 200)	@500: 403.15 > 346.60 → @700: 421.35 < 485.03	Identical CPU sort cost to fp32_plain but half HNSW index pages
FP16 Extended	P5	(4.0, 0.01)	12/18	6/18	**[1250, 1500]** (width 250)	@1250: 438.70 > 383.15 → @1500: 449.90 < 459.76	Linear inline Seq Scan + Sort cost overtakes unadjusted HNSW
FP16 Extended	P9	(8.0, 0.05)	12/18	6/18	**[1250, 1500]** (width 250)	@1250: 888.23 > 865.74 → @1500: 910.89 < 1038.79	High cpu_op (0.4922 N) + half HNSW index pages (375 vs 748)
FP16 Extended	P4	(4.0, 0.0025)	14/18	4/18	**[1750, 2000]** (width 250)	@1750: 454.83 > 409.72 → @2000: 462.90 < 468.24	Linear inline Seq Scan + Sort (0.2341 N) overtakes unadjusted HNSW
FP16 Extended	P8	(8.0, 0.01)	16/18	2/18	**[3000, 4000]** (width 1000)	@3000: 971.34 > 919.42 → @4000: 1001.47 < 1225.85	Linear inline Seq Scan + Sort overtakes rpc = 8.0 unadjusted HNSW
FP16 Extended	P7	(8.0, 0.0025)	17/18	1/18	**[4000, 5000]** (width 1000)	@4000: 997.72 > 936.46 → @5000: 1053.94 < 1170.57	Final cell to converge (relpages 800 -> 1,000 at rpc = 8.0)

**Note: For fp32_extended P3, fp32_plain P1–P3, and fp16_extended P1–P3, HNSW is selected across all tested scales starting at the lower sweep bound N = 500; any crossover, if present, occurs below the tested sweep range (N < 500).*

Key mechanisms established by Tables II-B and II-C:

1. **The N = 786 → 787 `ratio < 0.5` Guard Transition (FP32 Extended):** For fp32_extended, startupPages > relpages holds at all 18 tested scales ($N \in$ [500, 5,000]), but ratio < 0.5 is False for $N \in$ [500, 786] (6/18 scales), keeping Δ = 0 and forcing Seq Scan in 8/9 profiles (P1, P2, P4–P9). Stepping 1 row from N = 786 (ratio = 0.500041) to N = 787 (ratio = 0.499493) activates the guard: at Profile P4, using consistent two-decimal EXPLAIN display values, fp32_extended HNSW startup cost drops by **88.06× on a same-scale basis** (from unadjusted 788.15 to adjusted 8.95, 788.15 / 8.95 = 88.0615×) and by **87.93× on a cross-scale basis** relative to N = 786 (787.01 / 8.95 = 87.9341×); using consistent unrounded double-precision values (788.150954 at N=787, 787.011542 at N=786, and 8.948258 adjusted at N=787), the reductions are **88.08× same-scale** (88.0787×) and **87.95× cross-scale** (87.9514×). Root Limit total cost drops from 797.45 (N = 786) to 29.31

(N = 787, below `Seq Scan + Sort` 32.87), flipping `7/9` profiles (`P1, P2, P4, P5, P6, P8, P9`) to `Index Scan`.

2. **Inline Crossover Dynamics, Discrete `relpages` Steps, and the N = 850 Fuzzy Tie-Break:** For `fp32_plain` and `fp16_extended`, `guard_active` is False across **all `18/18` scales (`0/162` cells each)**; their crossovers in Table II-C occur as linear `Seq Scan + Sort` cost overtakes logarithmic unadjusted HNSW cost. At high `cpu_operator_cost` (`P6, P9`), `fp16_extended` crosses over earlier than `fp32_plain` (`[500, 700]` vs. `[790, 850]` at `P6`; `[1250, 1500]` vs. `[1750, 2000]` at `P9`) because both pay identical per-tuple CPU sort costs while `fp16_extended` pays half the HNSW index startup page I/O. On `fp32_extended` at `P7`, discrete integer stepping of `relpages` (8 pages at N = 1,250 [`50.66 < 51.63`] jumping to `10` pages at N = 1,500 [`61.19 > 53.75`]) places the actual crossover in `[1250, 1500]`. Across all 486 cells, `485/486` select the strictly lower `total_cost`, and `1/486` (**`N = 850, fp32_plain, P6`**) is decided by PostgreSQL's `1%` `STD_FUZZ_FACTOR = 1.01` tie-break: HNSW root `Limit` total cost (`846.04`) and `Seq Scan + Sort` total cost (`843.86`) are within the `1%` total-cost fuzz factor (`846.04 <= 1.01 * 843.86 = 852.30`), after which the startup comparison compares HNSW startup cost (`814.52`) against raw `Seq Scan + Sort` startup cost (`843.36`) and selects HNSW because `814.52` is below the `1%` fuzzy startup threshold `843.36 / 1.01 = 835.01`.

C. *Client Round-Trip Latency, Outlier Sensitivity, and Statistical Inference (RQ2)*

Fig. 2 illustrates the 18-scale planner transition map alongside Profile P4 client latency scaling, and Table III summarizes the non-parametric statistical tests over N = 20 query blocks.

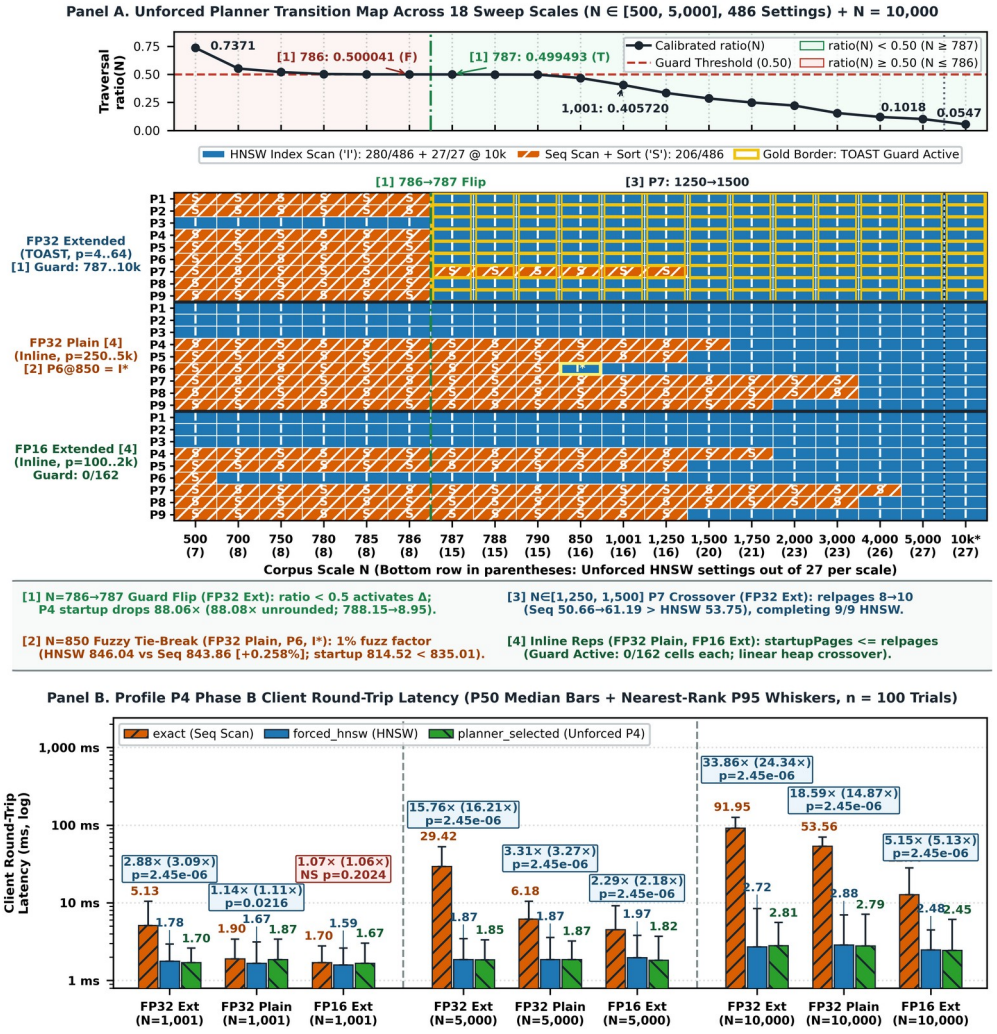


Fig. 2. Unforced Planner Transition Map Across 18 Sweep Scales (N ∈ [500, 5,000], 486 Settings) Plus N = 10,000 (27 Settings) (Panel A) and Profile P4 Client Round-Trip Latency Distribution (n = 100 Trials per Setting) (Panel B). In **Panel A**, the upper strip plots the calibrated HNSW entry-traversal fraction ratio(N) = (E_0 + 54.152441134 ln N)/N against the guard threshold ratio = 0.50, and the lower `27 × 19` (513-cell) matrix displays unforced plan choices (`'I'` = HNSW `Index Scan`, `'S'` = `Seq Scan + Sort`, `'I*'` = `1%` `STD_FUZZ_FACTOR` startup-cost tie-break; gold cell border = TOAST discount guard active). Across the 18-scale sweep (`486` settings), PostgreSQL selects HNSW in `280/486` (`57.6%`) settings and `Seq Scan + Sort` in `206/486` (`42.4%`), plus `27/27` (`100%`) at N = 10,000. Callout keys `[1]`–`[4]` in the external strip below Panel A document: **[1]** the discrete guard activation at N = 786 → 787 (`ratio: 0.500041 -> 0.499493`), which drops Profile P4 `FP32 Extended` HNSW startup cost by `88.06×` display (`88.08×` unrounded; `788.15 -> 8.95`) and root `Limit` total cost from `797.45` to `29.31`, flipping `7/9` profiles to HNSW; **[2]** the single fuzzy tie-break cell `'I*'` at N = 850, `FP32 Plain`, Profile P6 (`HNSW Limit = 846.04` vs. `Seq+Sort = 843.86` [`+0.258%` within the `1%` fuzz band `1.01 * 843.86 = 852.30`], where HNSW wins because its startup cost `814.52` is below raw `Seq+Sort` startup cost `843.36` and below the `1%` fuzzy startup threshold `843.36 / 1.01 = 835.01`); **[3]** the `FP32 Extended` Profile P7 crossover at N ∈ [1,250, 1,500] (`relpages: 8 -> 10`, pushing `Seq+Sort` from `50.66` to `61.19 > HNSW 53.75`); and **[4]** the complete absence of guard activation (`0/162` cells each) for inline `FP32 Plain` and `FP16 Extended` because `startupPages <= relpages` across all scales. **Panel B** reports Profile P4 client round-trip latency (`P50` median bars with nearest-rank `P95` upper whiskers across n = 100 measured trials after 2 warm-up passes [`40` warm-up executions across the 20 queries]), paired with `exact / forced_hnsw` median (and mean) speedup badges and Benjamini–Hochberg adjusted two-sided Wilcoxon signed-rank p-values across the 9-test family (α = 0.05).

TABLE III. NON-PARAMETRIC STATISTICAL INFERENCE SUMMARY OVER N = 20 QUERY-LEVEL EXPERIMENTAL UNITS

SUB-TABLE III-A: TIE-CORRECTED FRIEDMAN OMNIBUS TESTS ACROSS PROFILES P1–P9 (PLANNER_SELECTED MODE, DF = 8, C_{TIES} = 1.0000)

Scale (N)	Representation	Distinct Profile Split (HNSW / Seq Scan of 9)	Friedman Q Statistic	Raw p-Value (χ^2_8)	BH-Adjusted p_{adj}	Role & Inference (α = 0.05)
1,001	FP32 Extended	8 / 1 (160 / 20 captures)	50.547	3.21e-08	**2.89e-07 (< 0.001)**	Plan-Flip Sensitivity: **Significant**
1,001	FP32 Plain	4 / 5 (80 / 100 captures)	25.267	1.40e-03	**0.0063**	Plan-Flip Sensitivity: **Significant**
1,001	FP16 Extended	4 / 5 (80 / 100 captures)	15.547	0.0493	0.1480	Plan-Flip Sensitivity: Not Significant
5,000	FP32 Extended	9 / 0 (180 / 0 captures)	8.160	0.4180	0.6270	Negative Control (Identical Plan): NS
5,000	FP32 Plain	9 / 0 (180 / 0 captures)	6.000	0.6472	0.6860	Negative Control (Identical Plan): NS
5,000	FP16 Extended	9 / 0 (180 / 0 captures)	9.467	0.3045	0.5480	Negative Control (Identical Plan): NS
10,000	FP32 Extended	9 / 0 (180 / 0 captures)	5.653	0.6860	0.6860	Negative Control (Identical Plan): NS
10,000	FP32 Plain	9 / 0 (180 / 0 captures)	6.720	0.5671	0.6860	Negative Control (Identical Plan): NS
10,000	FP16 Extended	9 / 0 (180 / 0 captures)	10.507	0.2313	0.5203	Negative Control (Identical Plan): NS

SUB-TABLE III-B: EXPLORATORY PAIRED TWO-SIDED WILCOXON SIGNED-RANK TESTS COMPARING REPRESENTATIONS UNDER PROFILE P4 FORCED_HNSW (HOLM STEP-DOWN ADJUSTED, CONTINUITY-CORRECTED ASYMPTOTIC NORMAL & EXACT COMBINATORIAL)*

Scale (N)	Paired Representation Comparison (forced_hnsw at P4)	Wilcoxon W	Asymptotic Raw p / Holm p_{adj}	Exact Combinatorial Raw p / Holm p_{adj}	Matched Rank-Biserial r_{rb}	Inference (α = 0.05)
1,001	FP16 Extended vs. FP32 Extended	55.0	0.0646 / 0.1938	0.0637 / 0.1912	-0.476	Not Significant
1,001	FP16 Extended vs. FP32 Plain	43.0	0.0217 / 0.0888	0.0192 / 0.0769	-0.590	Not Significant
5,000	FP16 Extended vs. FP32 Extended	98.0	0.8083 / 1.0000	0.8124 / 1.0000	+0.067	Not Significant
5,000	FP16 Extended vs. FP32 Plain	91.0	0.6143 / 1.0000	0.6215 / 1.0000	+0.133	Not Significant
10,000	FP16 Extended vs. FP32 Extended	41.0	0.0178 / 0.0888	0.0153 / 0.0766	-0.610	Not Significant
10,000	FP16 Extended vs. FP32 Plain	28.0	0.00429 / **0.0257**	0.00271 / **0.0163**	**-0.733**	**Significant (Exploratory)***

**Note: Inter-representation `forced_hnsw` latency comparisons are exploratory and conditional on a single benchmark session, a single HNSW graph build per table, and the 20 fixed queries (`q0`–`q19`), where coordinate precision, operator class (`halfvec_cosine_ops` vs. `vector_cosine_ops`), HNSW index size (`20,398,080 B` = `20.40 MB` decimal / `19.45 MiB` binary vs. `40,779,776 B` = `40.78 MB` decimal / `38.89 MiB` binary), and heap layout vary together.*

SUB-TABLE III-C: PROFILE P4 CLIENT ROUND-TRIP LATENCIES (N = 100 TRIALS), DIAGNOSTIC `EXPLAIN ANALYZE` EXECUTION TIMES (N = 20 RUNS), AND EXACT VS. FORCED-HNSW CLIENT INFERENCE (N = 20 QUERIES)*

Scale (N)	Representation	Client `exact` P_{50} / P_{95} / Mean ± SD (ms)	Client `forced_hnsw` P_{50} / P_{95} / Mean ± SD (ms)	Client `planner_selected` P_{50} / P_{95} / Mean ± SD (ms)	Diagnostic `Execution Time` `exact` / `forced_hnsw` P_{50} (ms)*	Client Median Ratio (P_{50})	Client Mean Ratio & 95% Bootstrap CI	Continuity-Corrected Asymptotic BH p_{adj} / Exact BH p_{adj}	Inference (α = 0.05)
1,001	FP32 Extended	5.127 / 10.475 / 6.171 ± 6.575	1.779 / 2.964 / 1.998 ± 1.134	1.701 / 2.625 / 1.825 ± 0.374	10.489 / 0.348	**2.88×**	3.09× [2.39×, 4.13×]	1.23e-04 / **2.45e-06**	**Significant**
1,001	FP32 Plain	1.899 / 3.420 / 2.131 ± 0.606	1.670 / 3.142 / 1.917 ± 0.677	1.870 / 3.411 / 2.103 ± 0.585	0.576 / 0.328	**1.14×**	1.11× [1.04×, 1.20×]	0.0244 / **0.0216**	**Significant**
1,001	FP16 Extended	1.699 / 2.788 / 1.845 ± 0.430	1.591 / 2.627 / 1.736 ± 0.420	1.674 / 3.045 / 1.857 ± 0.513	0.380 / 0.249	**1.07×**	1.06× [0.99×, 1.14×]	0.1978 / **0.2024**	Not Significant
5,000	FP32 Extended	29.416 / 53.056 / 36.027 ± 13.128	1.866 / 3.480 / 2.223 ± 1.140	1.853 / 3.358 / 2.080 ± 0.588	44.172 / 0.680	**15.76×**	16.20× [14.07×, 18.53×]	1.23e-04 / **2.45e-06**	**Significant**
5,000	FP32 Plain	6.182 / 10.475 / 6.968 ± 2.752	1.867 / 3.582 / 2.128 ± 0.660	1.870 / 3.244 / 2.127 ± 0.639	6.755 / 0.469	**3.31×**	3.27× [2.92×, 3.70×]	1.23e-04 / **2.45e-06**	**Significant**
5,000	FP16 Extended	4.504 / 9.176 / 4.883 ± 2.021	1.971 / 3.835 / 2.241 ± 0.803	1.824 / 3.728 / 2.164 ± 0.910	4.909 / 0.485	**2.29×**	2.18× [1.95×, 2.48×]	1.23e-04 / **2.45e-06**	**Significant**
10,000	FP32 Extended	91.954 / 126.147 / 93.638 ± 24.100	2.716 / 8.466 / 3.847 ± 4.127	2.811 / 5.596 / 3.150 ± 1.500	96.708 / 1.179	**33.86×**	24.34× [19.13×, 30.72×]	1.23e-04 / **2.45e-06**	**Significant**
10,000	FP32 Plain	53.556 / 70.385 / 52.800 ± 10.927	2.881 / 7.020 / 3.550 ± 1.889	2.785 / 7.157 / 3.433 ± 1.919	25.721 / 1.281	**18.59×**	14.87× [13.30×, 16.81×]	1.23e-04 / **2.45e-06**	**Significant**
10,000	FP16 Extended	12.770 / 28.163 / 14.252 ± 6.974	2.479 / 4.499 / 2.776 ± 1.053	2.450 / 6.125 / 2.906 ± 1.456	11.415 / 0.798	**5.15×**	5.13× [4.45×, 5.86×]	1.23e-04 / **2.45e-06**	**Significant**

**Note: Diagnostic `Execution Time` values include per-node `EXPLAIN ANALYZE` timing instrumentation overhead (`TIMING ON` default) from a separate 1-pass diagnostic run (`[Supplementary Material – Repository/DOI Placeholder]`) and are reported strictly as diagnostic measurements, whereas all speedup ratios, bootstrap CIs, and Wilcoxon tests are computed from the 5-pass client round-trip latency trials.*

1. **Client Round-Trip Speedups and Benign N = 1,001 Inline `FP16 Extended` Fallback (Sub-Table III-C):** Eight of nine Profile P4 `exact` vs. `forced_hnsw` comparisons are statistically significant (`p_adj = 2.45e-06` exact / `1.23e-04` continuity-corrected asymptotic normal for the 7 rows with W = 0.0, and `0.0216` exact / `0.0244` continuity-corrected asymptotic for `1,001 fp32_plain`). The sole non-significant contrast is **`fp16_extended` at N = 1,001** (`1.699 ms` `exact` vs. `1.591 ms` `forced_hnsw` and `1.674 ms` `planner_selected` P_{50}; client median ratio `1.07×`, mean ratio `1.06×`, exact BH p_{adj} = 0.2024), confirming that the optimizer's choice of `Seq Scan` on `fp16_extended` at N = 1,001 (P4) causes no statistically significant client-latency degradation. In our exploratory inter-representation comparison under P4 `forced_hnsw` (Sub-Table III-B), `fp16_extended` exhibits a lower client latency than `fp32_plain` at N = 10,000 (2.479 ms vs. 2.881 ms P_{50}; W = 28.0, exact Holm p_{adj} = 0.0163, asymptotic 0.0257, r_{rb} = -0.733). Because this contrast is conditional on one benchmark session, one HNSW index build per table, and 20 fixed queries—and simultaneously varies coordinate precision (`halfvec` vs. `vector`), operator class (`halfvec_cosine_ops` vs. `vector_cosine_ops`), HNSW index footprint (`20,398,080 B` = `20.40 MB` decimal / `19.45 MiB` binary vs. `40,779,776 B` = `40.78 MB` decimal / `38.89 MiB` binary), and heap tuple density (5 vs. 2 tuples/page)—it must be interpreted strictly as an exploratory observation of the combined configuration rather than isolated evidence that FP16 precision itself causes a latency improvement.

2. **Harmful `FP32 Extended` Profile `P7` Fallback at N = 1,001 and Full Dataset Outlier Distribution:** At N = 1,001, `fp32_extended` under Profile `P7` (`planner_selected`, `Seq Scan`, n = 100 trials) records a **client median (P_{50}) of `4.9350 ms` versus `1.7010 ms` at P4 (`2.90×` median slowdown**; `2.87×–2.97×` across all eight HNSW-selecting profiles), and an overall sample mean of **`11.6677 ms` with sample SD `34.6999 ms`** (`6.39×` mean ratio vs. `P4 1.8247 ms`). At the paired query level across all n = 20 evaluation queries (5 measured passes per query in `[Supplementary Material – Repository/DOI Placeholder]`), `P7` (`Seq Scan`) is slower than `P4` (`Index Scan`) on **`20/20` queries** under both per-query mean aggregation (paired difference $\Delta_{\text{mean, q}} = \bar{L}_{P7}(q) - \bar{L}_{P4}(q)$ ranging from `+2.7167 ms` on `q6` to `+60.4551 ms` on `q1`, mean difference `+9.8431 ± 14.6478 ms`, median difference `+3.8344 ms` [IQR `3.3922 ms`], two-sided Wilcoxon W = 0.0, exact combinatorial $p = 1.907 \times 10^{-6}$, continuity-corrected asymptotic normal $p = 9.569 \times 10^{-5}$ [z = -3.9013, standardized $r = |z|/\sqrt{(20)} = 0.8723$; uncorrected normal $p = 8.857 \times 10^{-5}$, r = 0.8765], matched rank-biserial r_{rb} = +1.0000) and per-query median aggregation (paired difference $\Delta_{\text{med, q}} = \tilde{L}_{P7}(q) - \tilde{L}_{P4}(q)$ ranging from `+1.4987 ms` on `q6` to `+9.1075 ms` on `q10`, mean difference `+3.5945 ± 1.4974 ms`, median difference `+3.3266 ms` [IQR `1.0125 ms`], W = 0.0, exact two-sided $p = 1.907 \times 10^{-6}$, continuity-corrected asymptotic normal $p = 9.569 \times 10^{-5}$ [r = 0.8723; uncorrected $p = 8.857 \times 10^{-5}$, r = 0.8765], r_{rb} = +1.0000). Pass-by-pass inspection (`[Supplementary Material – Repository/DOI Placeholder]`) shows that Passes 1, 3, 4, and 5 have means of `5.7702 ms`, `5.0446 ms`, `5.7363 ms`, and `5.6624 ms` (`0` spikes > 50 ms), whereas Pass 2 (`mean = 36.1251 ms`, `median = 5.9165 ms`) contains four spikes > 50 ms within a `1.555 s` window (`181.5276 ms` on `q17`, `53.1563 ms` on `q3`, `78.7237 ms` on `q10`, and

294.2680 ms on q1). Without deleting raw data, sensitivity analysis establishes:

- **Excluding the 4 Pass-2 spikes (*n* = 96):** Mean = **5.8239 ms**, Sample SD = **3.2329 ms**, Median = 4.9118 ms (**3.19× mean ratio**, **2.89× median ratio** vs. P4, closely matching the P4 forced exact mean of 6.1707 ms).
- **Excluding only the single maximum (294.2680 ms, *n* = 99):** Mean = **8.8132 ms**, Sample SD = **19.8296 ms** (**4.83× mean ratio**).
- **Excluding Pass 2 entirely (*n* = 80):** Mean = **5.5534 ms**, Sample SD = **2.4823 ms**, Median = 4.9118 ms (**3.04× mean ratio**).

Across the entire 9,900-trial client-latency dataset ([Supplementary Material — Repository/DOI Placeholder]), exactly **178 observations exceed 50 ms**: 171 occur at *N* = 5,000 and 10,000 in exact (Seq Scan) mode (17 in 5,000 fp32_extended exact P4, 97 in 10,000 fp32_extended exact P4, and 57 in 10,000 fp32_plain exact P4, where full-table scans inherently require 50–193 ms); 5 occur at *N* = 1,001 during Pass-2 fp32_extended Seq Scan execution (the 4 P7 spikes above plus 1 trial in P4 exact on q7 at 67.6575 ms, 0.44 s after the P7 q1 spike); and 2 occur during HNSW execution under Profile P3 (80.4967 ms on 5,000 fp32_plain P3 Pass 1 q13, and 75.7794 ms on 10,000 fp16_extended P3 Pass 3 q16).

D. Primary Conventional *Recall@10* Summary (RQ3)

Evaluating our primary retrieval metric—conventional ID-intersection Recall@10 against the exhaustive exact FP32 reference set ([Supplementary Material — Repository/DOI Placeholder])—establishes two core findings. Table IV and Fig. 3 (in Appendix A) report the complete query-level Recall@10 decomposition, 10,000-iteration nonparametric bootstrap 95% confidence intervals (B = 10,000, random.Random(42) reset per series), 19-query sensitivity excluding Q9, and secondary τ-tolerance tie-aware analysis:

1. **Exact Sequential Scan (exact mode, *n* = 20 queries):** End-to-end 16-bit fp16_extended matches exact FP32 top-10 rankings on **599/600 candidate slots** with zero graph variance: **1.0000 (95% CI [1.0000, 1.0000])** at *N* = 1,001, **0.9950 (95% CI [0.9850, 1.0000], 199/200 slots; 19/20 queries identical, with a single 1-slot swap on Q17 across a Δ *d* = 1.665 × 10^{-6} gap)** at *N* = 5,000, and **1.0000 (95% CI [1.0000, 1.0000])** at *N* = 10,000.

2. **Forced HNSW (forced_hnsw at Profile P4, *n* = 20 queries):** Primary 20-query conventional Recall@10 (with 10,000-replication query-level bootstrap percentile 95% CIs) is **0.9850 [0.9700, 1.0000]** across all three representations at *N* = 1,001; **0.9600 [0.9050, 1.0000]** (fp16_extended) versus **0.9550 [0.9000, 1.0000]** (fp32_extended and fp32_plain) at *N* = 5,000; and **0.9350 [0.8450, 0.9950]** (fp16_extended and fp32_extended) versus **0.9300 [0.8350, 0.9950]** (fp32_plain) at *N* = 10,000. As detailed in Table IV and Fig. 3 (Appendix A), the apparent drop to 0.9350 at *N* = 10,000 is driven by a single query (Q9), where 28 corpus chunks are **duplicate embedding vectors equal under pgvector's vector equality operator (=)** (378/378 pairs return True under a.embedding = b.embedding with cosine distance 0.0) competing for 8 boundary slots (0.9737 [0.9368, 1.0000] conventional Recall@10 across the remaining 19 queries, and 0.9750 equivalence-class Tie-Aware Recall@10).

VI. Discussion, Dimensionality Regimes, Practical Guidelines, and Limitations

A. Guard Boundaries, Analytical Extrapolation, and Dimensionality Regimes

Our 18-scale sweep and source-code analysis highlight three structural properties of pgvector 0.8.4's startupPages > relpages && ratio < 0.5 heuristic:

1. **Parameter Dependence of the Lower ratio < 0.5 Boundary (*N* = 786 → 787):** Because ratio(*N*) in (3) depends only on *N*, *m*, and ef_{search}, the lower guard activation at *N* = 786 → 787 is invariant to vector dimension *d* but shifts whenever *m* or hnsw.ef_search is modified (e.g., increasing ef_search above 40 increases ratio and pushes the ratio < 0.5 threshold to a larger *N*).

2. **Analytical Projection of the Upper Guard Boundary (*N* ≈ 50,000 on FP32 Extended):** Across our experimentally tested scales ($N \le 10{,}000$), startupPages > relpages holds for fp32_extended at every scale (253.90 > 32 at *N* = 5,000, and ~270 > 64 at *N* = 10,000). However, because startupPages(*N*) ≈ 0.4985 (48 + 54.152441ln *N*) grows logarithmically while main-heap relpages(*N*) ≈ *N* / 157 grows linearly, equating the two predicts an upper analytical crossing near ***N* ≈ 50,000 (~49.5k–50.2k)**, above which relpages exceeds startupPages and deactivates *Δ* even on TOASTed fp32_extended. We emphasize that *N* = 10,000 is our maximum experimentally tested scale and that this *N* ≈ 50,000 comparison is strictly an **analytical projection** beyond the measured scale range: under explicit projection assumptions at *N* = 50,000 (entryLevel(50,000) = $\lfloor \ln 50{,}000 / \ln 16 \rfloor = 3$, C_{cal} = 54.152441134, ratio(50,000) = 0.01267835, numIndexPages ≈ 0.4985 *N* = 24,925, and relpages ≈ $\lceil N/157 \rceil = 319$ > startupPages = 316.01 so Δ = 0), unadjusted HNSW Limit total cost remains well below Seq Scan + Sort under canonical Profile P4 (random_page_cost = 4.0, cpu_operator_cost = 0.0025: 1,288.93 vs. 2,024.51) and across 8/9 profiles (P1–P6, P8, P9), whereas only the extreme high-random-cost sensitivity probe P7 (random_page_cost = 8.0, cpu_operator_cost = 0.0025) projects a higher unadjusted HNSW Limit cost (2,572.90 vs. 2,024.51) upon guard deactivation.

3. **Dimensionality Regimes Across Standard Embedding Sizes (TOAST_TUPLE_THRESHOLD = 2,032 B):** Comparing uncompressed datum size (VARSIZE(*d*) = 8 + bytes_per_elem· *d*) and raw two-column tuple length ($t_{len}(d)$ = 28 + VARSIZE(*d*), with 8-byte aligned page-slot footprint MAXALIGN($t_{len}(d)$) = 32 + VARSIZE(*d*)) against the default 2,032 B threshold (TOAST_TUPLE_THRESHOLD / default toast_tuple_target) defines three qualitative dimensionality regimes (~500-D transition for vector(d) and ~1,000-D transition for halfvec(d)). Because exact single-dimension integer cutoffs near *d* ≈ 500 and *d* ≈ 1,000 depend on table-specific non-vector columns and tuple header layout (t_hoff), Table V organizes the three regimes around standard embedding model dimensions ($d \in \{384, 512, 768, 1{,}024, 1{,}536\}$), highlighting the empirically verified split regime at *d* = 768.

TABLE V. ANALYTICAL DIMENSIONALITY REGIMES FOR `VECTOR(D)` VS. `HALFVEC(D)` UNDER `STORAGE EXTENDED` (`TOAST_TUPLE_THRESHOLD = 2,032 B`)

Dimensionality *d* (Representative Same-*d* Models)*	`vector(d)` Datum / Raw t_{len} / `MAXALIGN` (B)*	`vector(d)` `EXTENDED` Layout & Tuples/Page	`halfvec(d)` Datum / Raw t_{len} / `MAXALIGN` (B)*	`halfvec(d)` `EXTENDED` Layout & Tuples/Page	Resulting `startupPages > relpages` Regime at $N \approx 1{,}000$
***d* = 384** (`all-MiniLM-L6-v2` — unbenchmarked example)	`1,544 / 1,572 / 1,576`	**Inline** (5 tuples/page, `relpages ≈ 201`)	`776 / 804 / 808`	**Inline** (10 tuples/page, `relpages ≈ 101`)	**Regime I (Sub-threshold band, *d* < ~500):** Both FP32 and FP16 stay well below `2,032 B` and fit **inline** (`guard_active = False` for both).
***d* = 512** (`CLIP-ViT-B/32` — unbenchmarked example)	`2,056 / 2,084 / 2,088`	**TOASTed** (~157 ptrs/page, `relpages ≈ 7`)	`1,032 / 1,060 / 1,064`	**Inline** (7 tuples/page, `relpages ≈ 143`)	**Regime II (Split band, ~500 < *d* < ~1,000): Split regime**—FP32 exceeds `2,032 B` (`guard_active = True` for $N \geq 787$), FP16 stays **inline** (`guard_active = False`).
***d* = 768** (**Benchmarked**: `nomic-embed-text`; unbenchmarked examples: `bge-base`, `e5-base`)	`3,080 / 3,108 / 3,112` (`3,076` ext)	**TOASTed** (~157 ptrs/page, `relpages = 7` at 1,001)	`1,544 / 1,572 / 1,576`	**Inline** (5 tuples/page, `relpages = 201` at 1,001)	**Regime II (Split band): Empirically verified split regime**—FP32 is 100% TOASTed (`3,108 B > 2,032 B`), FP16 is 100% inline (`1,572 B <= 2,032 B`).
***d* = 1,024** (`bge-large`, `e5-large` — unbenchmarked examples)	`4,104 / 4,132 / 4,136`	**TOASTed** (~157 ptrs/page, `relpages ≈ 7`)	`2,056 / 2,084 / 2,088`	**TOASTed** (~157 ptrs/page, `relpages ≈ 7`)	**Regime III (Super-threshold band, *d* > ~1,000):** Both exceed `2,032 B` and move to `pg_toast` (`guard_active = True` for both at $N \geq 787$).
***d* = 1,536** (`text-embedding-3-small` — unbenchmarked example)	`6,152 / 6,180 / 6,184`	**TOASTed** (~157 ptrs/page, `relpages ≈ 7`)	`3,080 / 3,108 / 3,112`	**TOASTed** (~157 ptrs/page, `relpages ≈ 7`)	**Regime III (Super-threshold band):** Both are TOASTed; keeping `halfvec(1536)` inline requires explicit `STORAGE PLAIN` (2 tuples/page).

**Note: Only `nomic-embed-text` (d = 768) was experimentally benchmarked in this study; other listed models are unbenchmarked examples illustrating each dimensionality band. For the two-column `(chunk_id INT, embedding)` schema (`t_hoff = 24 B`), datum size is VARSIZE(d) = 8 + bytes_per_elem· d, raw tuple length is $t_{len}(d) = 28 + VARSIZE(d)$, and 8-byte aligned page-slot footprint is $MAXALIGN(t_{len}(d)) = 32 + VARSIZE(d)$ (plus `4 B ItemIdData` per page slot); exact boundary dimensions near d ≈ 500 and d ≈ 1,000 vary with non-vector table columns.*

B. Practical Guidelines and Methodological Limitations

Practical Guidelines: (1) For 768-D embeddings (`Regime II`), adopting `halfvec(768)` (`fp16_extended`) cuts total storage by `55.21%–55.37%` and HNSW index size by `~50%`, packs 5 inline tuples/page, stores no out-of-line TOAST chunk data (`toast_table_bytes = 0 B`, `0` chunk pages) for the tested `fp16_extended` representation, and stays below the `4,096`-block `BAS_BULKREAD` threshold at $N = 10{,}000$; (2) when migrating a small 768-D table ($N < 2{,}000$) to `halfvec(768)` under Profile `P4`, the resulting `Seq Scan` selection is benign in warm cache (`1.674 ms` vs. `1.591 ms` P_{50} at $N = 1{,}001$, NS) and automatically switches to HNSW at $N \geq 2{,}000$; and (3) on TOASTed `fp32_extended` tables ($m = 16$, $ef_{search} = 40$), the unforced planner selects `Seq Scan` across `8/9` tested cost profiles (`P1, P2, P4–P9`) when $N \leq 786$ (where ratio(N) ≥ 0.5 keeps the TOAST discount inactive) and under elevated `random_page_cost = 8.0` with default `cpu_operator_cost = 0.0025` (`P7`) up to $N = 1{,}250$, triggering full-table detoasting (`relpages + 3N` buffer hits and a `2.90×` client median slowdown at $N = 1{,}001$) unless index scans are enforced or cost parameters are adjusted.

Methodological Limitations: (1) *No Engine Patch:* We study unmodified PostgreSQL `18.4` and `pgvector` `0.8.4` with parallelism disabled (`max_parallel_workers_per_gather = 0`); (2) *Single-Session Warm Cache & Instrumentation:* Client round-trip timings reflect single-session warm-cache execution (`prepare=False`) with an inferred `1.11–1.64 ms` client-minus-server (`Planning + Execution`) difference (`1.19–1.68 ms` relative to `Execution Time` alone) across P4 `forced_hnsw` cells and transient Pass-2 spikes, while diagnostic-stage timings include `EXPLAIN ANALYZE` per-node timing overhead. Latency measurements characterize the observed benchmark runs; consistency of latency distributions across independent reruns was not established; (3) *Single HNSW Build & 20 Fixed Queries:* Each table uses one HNSW build ($m = 16$, $ef_{construction} = 64$); because repeated graph builds were not evaluated, the contribution of graph-construction variability to the observed `0.005` recall differences under `forced_hnsw` cannot be isolated, and inferences are conditioned on the 20 fixed queries (`q0`–`q19`); (4) *Duplicate-Vector Cluster:* The 28-chunk exact duplicate vector cluster in the prepared `BeIR/msmarco` slice drives the `Q9` boundary tie at $N = 10{,}000$ (Appendix A); (5) *Empirical Scale, Dimension Scope, and Nested-Prefix Sampling:* Empirical runs span $N \leq 10{,}000$ at $d = 768$ (while $N \approx 50{,}000$ and Table V are analytical projections), and because all scales are constructed via deterministic nested prefix selection (`ORDER BY chunk_id ASC LIMIT N`) rather than independent random subsampling, corpus composition and local vector neighborhood density shift with N (e.g., the recurring duplicate-vector cluster grows from 4 chunks at $N = 1{,}001$ to 12 at $N = 5{,}000$ and 28 at $N = 10{,}000$), so trends across N reflect both relation cardinality and evolving prefix composition rather than isolating scale alone; (6) *Coupled Representation Variables:* Contrasting `fp16_extended` and `fp32_plain` simultaneously varies precision, tuple density (5 vs. 2 rows/page), index size, and operator class without hardware performance counters; and (7) *Supplementary Evidence Archive and From-Scratch Reconstruction Boundaries:* All provisioning, benchmark, statistical-analysis, bootstrap, tie-sensitivity, and figure-generation scripts together with all captured `EXPLAIN` plan trees, storage catalogs, client-latency logs, buffer diagnostics, and verification manifests (45 scripts and dataset artifacts across the 3-scale evaluation, 18-scale sweep, 486-cell exact-mode cost validation, tie-sensitivity analysis, and publication figure suite) are archived in `[Supplementary Material — Repository/DOI Placeholder]`, enabling offline reproduction of the post-

collection cost-model calculations, statistical tests, bootstrap confidence intervals, tables, and figures from the preserved evidence. This does not establish complete from-scratch reproduction of corpus construction, embedding generation, or the original database state. Full from-scratch corpus construction and re-embedding require six additional upstream inputs not included in the post-collection archive: the raw upstream MS MARCO passage TSV (SHA-256 `cdca54aa...34e8b`), the upstream selector filter that sampled the `5,457` source documents, the embedding-generation module (and confirmation of any `nomic-embed-text` task prefixes), the initial FP32 vector table dump (`public.chunks_10k_fp32`), environment dependency lockfiles, and the exact Ollama model digest.

VII. Conclusion

We presented a closed-form C-source-to-`EXPLAIN` cost-model reconstruction and a multi-stage empirical evaluation (`4,860` baseline plan captures across `81` distinct settings, `972` 18-scale sweep captures plus `486` exact-mode validation captures across `486` settings, `9,900` timed client executions, and `540` buffer diagnostics) showing how FP16 vector representation (`halfvec(768)`) and PostgreSQL TOAST layouts (`STORAGE EXTENDED` vs. `STORAGE PLAIN`) govern query plan selection in PostgreSQL `18.4` with `pgvector 0.8.4`. Across $N \in [500, 5{,}000]$, `pgvector`'s intentional TOAST-compensation guard (`startupPages > relpages && ratio < 0.5`) activates for out-of-line `fp32_extended` at $N = 786 \rightarrow 787$—where the isolated startup-cost dampening factor is `88.06×` display / `88.08×` unrounded (`788.15 -> 8.95` same-scale at P4; `87.93×` display / `87.95×` unrounded cross-scale) and root `Limit` total cost drops from `797.45` to `29.31`—flipping `7/9` profiles to HNSW while leaving `P7` in a harmful `Seq Scan` (`2.90×` client median slowdown at N = 1,001) until `[1250, 1500]`. Conversely, inline `fp16_extended` (5 tuples/page, `relpages = 201` at N = 1,001) and `fp32_plain` (2 tuples/page, `relpages = 501`) keep `relpages >= startupPages` (`0/162` guard activations each), raising P4 root `Limit` total cost from `30.37` (`fp32_extended`) to `422.90` (`13.92×` display / `13.93×` unrounded) and `844.90` (`27.82×`) at N = 1,001 and causing `5/9` profiles each to select `Seq Scan` (benign on `fp16_extended` at P4, `1.07×` median ratio, $p_{adj} = 0.2024$) before linear `Seq Scan + Sort` cost growth restores `27/27` HNSW selection by N = 5,000. Separating access-method selection from coordinate precision, HNSW index traversal delivers statistically significant client-latency speedups over exact sequential scans at $N \geq 5{,}000$ across all three representations (`15.76×` and `33.86×` client median speedups on TOASTed `fp32_extended`, `3.31×` and `18.59×` on inline `fp32_plain`, and `2.29×` and `5.15×` on inline `fp16_extended` at N = 5,000 and 10,000, exact BH $p_{adj} = 2.45 \times 10^{-6}$, where the smaller ratio on `fp16_extended` reflects its faster inline exact scan rather than slower HNSW execution). Meanwhile, end-to-end `halfvec(768)` (`fp16_extended`) specifically cuts total relation storage by `55.21%–55.37%` and HNSW index size by `49.90%–49.98%`, stores no out-of-line TOAST chunk data for the tested `fp16_extended` representation (`toast_table_bytes = 0 B`, `0` chunk pages; rendering the N = 1,001 Profile P4 `Seq Scan` fallback benign at `1.674 ms` vs. `1.591 ms` P_{50}), and achieves `1.000 / 0.995 / 1.000` conventional `Recall@10` under exact scan (reflecting a 1-slot swap on `Q17` at N = 5,000) and `0.985 / 0.960 / 0.935` under P4 `forced_hnsw` versus `0.985 / 0.955 / 0.935` on `fp32_extended` and `0.985 / 0.955 / 0.930` on `fp32_plain` (`0.9737` and `0.9684` at N = 10,000 excluding `Q9`).

These findings apply within the boundary conditions of unmodified PostgreSQL `18.4` and `pgvector 0.8.4`, single-worker warm-cache execution (`relallvisible = 0`), a single HNSW graph build per table (m = 16, $ef_{construction}$ = 64, ef_{search} = 40), and 20 fixed topical `BeIR/msmarco` queries at d = 768 ($N \leq 10{,}000$). For database administrators operating in the split dimensionality band (`Regime II`, ~500 < d < ~1,000, empirically verified at d = 768), `halfvec(768)` stores no out-of-line TOAST chunk data for the tested `fp16_extended` representation (`toast_table_bytes = 0 B`, `0` chunk pages) and avoids `relpages + 3N` sequential-scan detoasting while remaining benign during small-table `Seq Scan` fallbacks; meanwhile, on `vector(768)` tables under `STORAGE EXTENDED` (m = 16, ef_{search} = 40), unforced sequential scans occur when $N \leq 786$ (ratio(N) ≥ 0.5 across `P1, P2, P4–P9`) or when `random_page_cost = 8.0` is paired with default `cpu_operator_cost = 0.0025` (`P7` at $N \leq 1{,}250$), where full-table detoasting degrades latency unless HNSW traversal is enforced or cost parameters are adjusted. For extension and optimizer designers, our source-level analysis demonstrates that keying TOAST compensation solely on main-heap `relpages` conflates compact inline representations with out-of-line TOASTed tables—and because PostgreSQL may allocate an empty `pg_toast` relation (`toast_table_bytes = 0 B`, `toast_index_bytes = 8,192 B`) even when no attributes are stored out-of-line, cost estimators should inspect effective per-column storage policy (`attstorage`) and actual out-of-line TOAST chunk usage (`pg_toast` chunk pages > 0) rather than mere `pg_toast` relation presence. Future work should extend this evaluation across multiple HNSW construction seeds, cold-cache NVMe I/O regimes, parallel query execution, and higher-dimensional embeddings ($d \geq 1{,}024$) where both FP32 and FP16 datums exceed `TOAST_TUPLE_THRESHOLD`.

Appendix A: Secondary Analysis — Retrieval Boundary Ties, Exact Vector-Equality Cluster, and τ-Tolerance Sensitivity

To support RQ3 without conflating distinct tie mechanisms or treating repeated planner profiles as independent queries, Table IV and Fig. 3 report query-level retrieval fidelity (n = 20 query-level observations per `(scale, representation, mode)` cell) along with 10,000-replication query-level percentile 95% bootstrap confidence intervals (`random.Random(42)` reset per series; `[Supplementary Material – Repository/DOI Placeholder]`). Because the 9 planner cost profiles (`P1`–`P9`) reuse the exact same 20 queries (`q0`–`q19`), we do not pool or bootstrap 9 × 20 = 180 profile-query combinations as if they were independent observations; instead, planner-selected profile splits are reported separately in the table note. Formally, for corpus slice C_N of cardinality N and query q, let $d_{FP32}(c, q)$ denote the exact FP32 cosine distance on `chunks_{N}_fp32_extended`, let $R_{ref}(q) = (c_{(1)}, \ldots, c_{(10)})$ denote the exact FP32 reference top-10 sequence returned by `ORDER BY distance ASC LIMIT 10` under `exact` (`Seq Scan`) mode, and let $D(q) = d_{FP32}(c_{(10)}, q)$ denote the exact FP32 rank-10 cutoff distance. Given any candidate top-10 result set $S \subseteq C_N$ ($|S| = 10$) returned by an evaluated configuration and a non-negative tolerance $\tau \geq 0$, we partition the reference boundary into the strictly closer set $B_\tau(q) = \{c \in C_N \mid d_{FP32}(c, q) < D(q) - \tau\}$ and the disjoint boundary-tie set $T_\tau(q) = \{c \in C_N \mid |d_{FP32}(c, q) - D(q)| \leq \tau\}$, with $r_\tau(q) = 10 - |B_\tau(q)|$ available boundary slots (`[Supplementary Material – Repository/DOI Placeholder]`). Conventional ID-intersection recall is $\text{Recall@10}_{conv}(S, q) = |S \cap R_{ref}(q)| / 10$, and tie-aware recall is $\text{Recall@10}_{tie}(S, q; \tau) = (|S \cap B_\tau(q)| + \min(|S \cap T_\tau(q)|, r_\tau(q))) / 10$. Because IEEE-754 single-precision (`binary32`) machine epsilon is $2^{-23} \approx 1.192 \times 10^{-7}$, $\tau = 10^{-7}$ and $\tau = 10^{-6}$ serve as operational numerical-equivalence thresholds

(with $\tau = 10^{-6}$ above FP32 machine epsilon); across our dataset, all 6 rank-10 boundary-tie instances at $\tau \in \{10^{-7}, 10^{-6}\}$ involve duplicate embedding vectors equal under `pgvector`'s `'='` operator with zero cosine distance difference ($\Delta\ d = 0.0$), whereas $\tau = 10^{-5}$ acts as an additive near-tie tolerance that additionally credits `Q17` at $N = 5{,}000$ ($\Delta\ d = 1.665 \times 10^{-6}$ around $D = 0.45295169$).

TABLE IV. QUERY-LEVEL RETRIEVAL DECOMPOSITION (N = 20 QUERIES): PRIMARY CONVENTIONAL RECALL@10 WITH 10,000-REPLICATION BOOTSTRAP 95% CIS (B = 10,000, RANDOM.RANDOM(42) RESET PER SERIES), 19-QUERY SENSITIVITY EXCLUDING Q9, AND SECONDARY TIE-AWARE RECALL@10 ($\tau \in \{10^{-7}, 10^{-6}, 10^{-5}\}$)

Scale (N)	Representation	Execution Mode*	Query Units (n)	Rank-10 Boundary Ties ($\lvert T\rvert > r$ at $\tau = 10^{-7}, 10^{-6} \mid 10^{-5}$)	Primary: Mean Conventional Recall@10 & 95% Bootstrap CI (20 Queries)	Sensitivity: Conventional Recall@10 & 95% CI Excl. Q9 (19 Queries)	Secondary: Equivalence-Class Tie-Aware Recall@10 ($\tau = 10^{-7}, 10^{-6}$)	Secondary: Tolerant Tie-Aware Recall@10 ($\tau = 10^{-5}$)	Tie Delta ($\tau = 10^{-6} \mid 10^{-5}$)
1,001	FP32 Extended	exact	20	3 / 20 (Q5, Q13, Q16)	**1.0000 [1.0000, 1.0000]**	1.0000 [1.0000, 1.0000]	1.0000	1.0000	+0.0000
1,001	FP32 Plain	exact	20	3 / 20 (Q5, Q13, Q16)	**1.0000 [1.0000, 1.0000]**	1.0000 [1.0000, 1.0000]	1.0000	1.0000	+0.0000
1,001	FP16 Extended	exact	20	3 / 20 (Q5, Q13, Q16)	**1.0000 [1.0000, 1.0000]**	1.0000 [1.0000, 1.0000]	1.0000	1.0000	+0.0000
1,001	FP32 Extended	forced_hnsw (P4)	20	3 / 20 (Q5, Q13, Q16)	**0.9850 [0.9700, 1.0000]**	0.9842 [0.9684, 1.0000]	0.9950	0.9950	+0.0100
1,001	FP32 Plain	forced_hnsw (P4)	20	3 / 20 (Q5, Q13, Q16)	**0.9850 [0.9700, 1.0000]**	0.9842 [0.9684, 1.0000]	0.9950	0.9950	+0.0100
1,001	FP16 Extended	forced_hnsw (P4)	20	3 / 20 (Q5, Q13, Q16)	**0.9850 [0.9700, 1.0000]**	0.9842 [0.9684, 1.0000]	0.9950	0.9950	+0.0100
5,000	FP32 Extended	exact	20	2 / 20 (Q3, Q9) | 3 / 20 (+Q17)	**1.0000 [1.0000, 1.0000]**	1.0000 [1.0000, 1.0000]	1.0000	1.0000	+0.0000
5,000	FP32 Plain	exact	20	2 / 20 (Q3, Q9) | 3 / 20 (+Q17)	**1.0000 [1.0000, 1.0000]**	1.0000 [1.0000, 1.0000]	1.0000	1.0000	+0.0000
5,000	FP16 Extended	exact	20	2 / 20 (Q3, Q9) | 3 / 20 (+Q17)	**0.9950 [0.9850, 1.0000]**	0.9947 [0.9842, 1.0000]	0.9950	1.0000	+0.0000 | +0.0050
5,000	FP32 Extended	forced_hnsw (P4)	20	2 / 20 (Q3, Q9) | 3 / 20 (+Q17)	**0.9550 [0.9000, 1.0000]**	0.9632 [0.9053, 1.0000]	0.9850	0.9850	+0.0300
5,000	FP32 Plain	forced_hnsw (P4)	20	2 / 20 (Q3, Q9) | 3 / 20 (+Q17)	**0.9550 [0.9000, 1.0000]**	0.9632 [0.9053, 1.0000]	0.9850	0.9850	+0.0300
5,000	FP16 Extended	forced_hnsw (P4)	20	2 / 20 (Q3, Q9) | 3 / 20 (+Q17)	**0.9600 [0.9050, 1.0000]**	0.9579 [0.9000, 1.0000]	0.9800	0.9850	+0.0200 | +0.0250
10,000	FP32 Extended	exact	20	1 / 20 (Q9)	**1.0000 [1.0000, 1.0000]**	1.0000 [1.0000, 1.0000]	1.0000	1.0000	+0.0000
10,000	FP32 Plain	exact	20	1 / 20 (Q9)	**1.0000 [1.0000, 1.0000]**	1.0000 [1.0000, 1.0000]	1.0000	1.0000	+0.0000
10,000	FP16 Extended	exact	20	1 / 20 (Q9)	**1.0000 [1.0000, 1.0000]**	1.0000 [1.0000, 1.0000]	1.0000	1.0000	+0.0000
10,000	FP32 Extended	forced_hnsw (P4)	20	1 / 20 (Q9)	**0.9350 [0.8450, 0.9950]**	**0.9737 [0.9368, 1.0000]**	0.9750	0.9750	+0.0400
10,000	FP32 Plain	forced_hnsw (P4)	20	1 / 20 (Q9)	**0.9300 [0.8350, 0.9950]**	**0.9684 [0.9211, 1.0000]**	0.9700	0.9700	+0.0400
10,000	FP16 Extended	forced_hnsw (P4)	20	1 / 20 (Q9)	**0.9350 [0.8450, 0.9950]**	**0.9737 [0.9368, 1.0000]**	0.9750	0.9750	+0.0400

**Note: Percentile 95% confidence intervals (`B = 10,000, random.Random(42) reset per series`) are computed by initializing an independent `random.Random(42)` instance for each `(scale, representation, mode, subset)` series, drawing n = 20 query-level observations (`n = 19` when excluding `Q9`, sorted by ascending integer `query_id`) with replacement via `rng.choice` across `10,000` replications, and taking the order-statistic bounds at indices `250` (`2.5%`) and `9,750` (`97.5%`) (`[Supplementary Material – Repository/DOI Placeholder]`). These intervals characterize uncertainty over the fixed 20-query workload (`q0`–`q19`) and do not establish generalization to the wider universe of semantic queries (degenerating to `[1.0000, 1.0000]` where all 20 queries score `1.0000`). Under unforced `planner_selected` mode across the 9 deterministic cost profiles (`P1`–`P9`), at N = 1,001, `fp32_extended` selects `exact` (`Seq Scan`) in `1/9` profiles (`P7`) and `HNSW` in `8/9` profiles, while `fp32_plain` and `fp16_extended` select `exact` in `5/9` profiles (`P4, P5, P7, P8, P9`) and `HNSW` in `4/9` profiles (`P1, P2, P3, P6`); at N = 5,000 and N = 10,000, all `9/9` profiles select HNSW `Index Scan` and therefore correspond identically to the `forced_hnsw` row.*

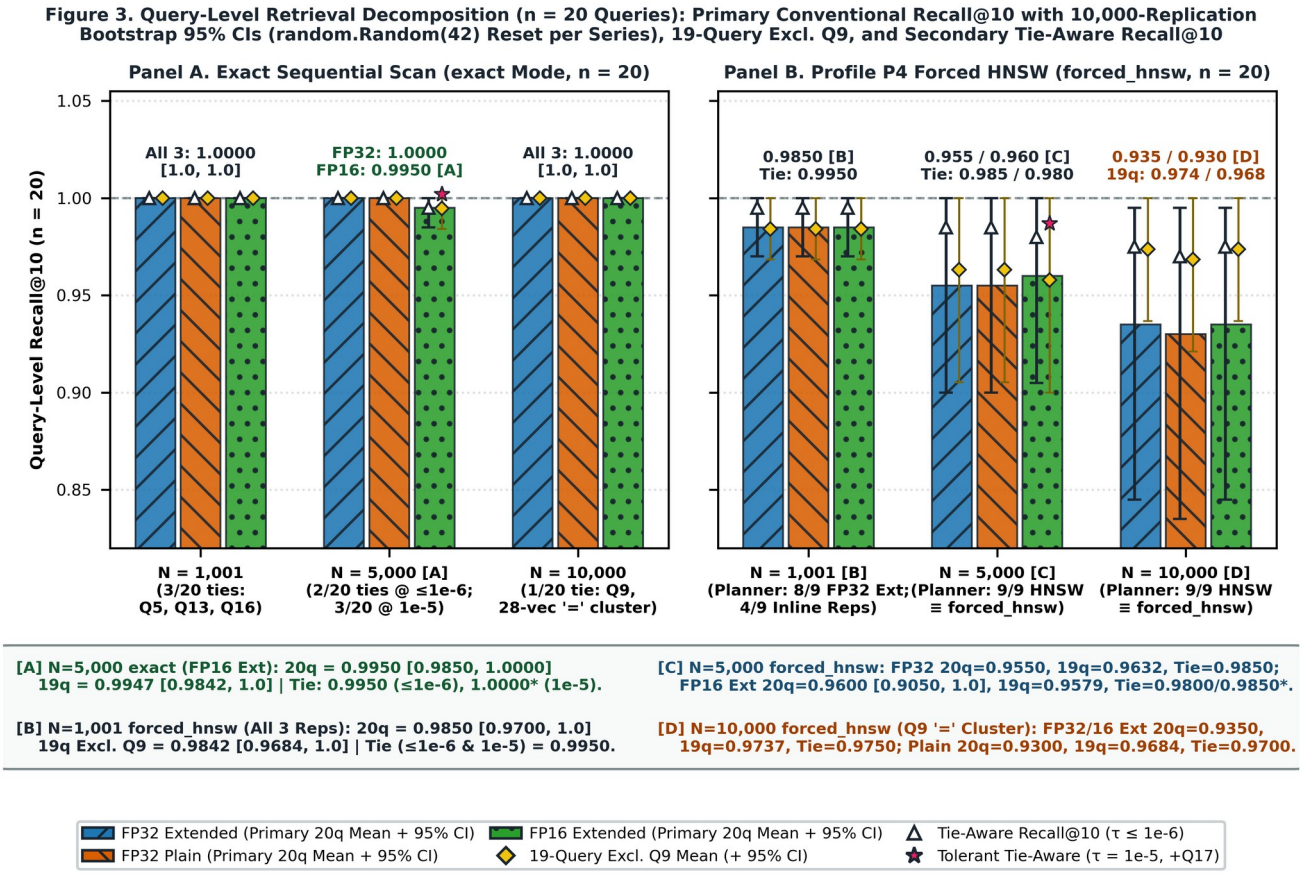


Fig. 3. Query-Level Retrieval Decomposition Across n = 20 Queries under Profile P4: Primary Conventional `Recall@10` (`20` Queries) with 10,000-Replication Nonparametric Percentile Bootstrap 95% Confidence Intervals (`B = 10,000, random.Random(42) reset per series`), 19-Query Sensitivity Excluding Boundary-Cluster Query `Q9`, and Secondary `Tie-Aware Recall@10` ($\tau \in \{10^{-7}, 10^{-6}, 10^{-5}\}$). **Panel A (`exact` Sequential Scan)** and **Panel B (`forced_hnsw` HNSW Index Scan)** plot primary 20-query conventional `Recall@10` bars with 95% bootstrap CIs (`sorted_means[250]` and `sorted_means[9750]` via `rng.choice`), overlaid with 19-query (`excl. Q9`) mean diamonds (`D`) + 95% CI whiskers, secondary tie-aware `Recall@10` upward triangles (`^`) at equivalence-class tolerance $\tau \le 10^{-6}$, and tolerant tie-aware stars (`*`) at $\tau = 10^{-5}$ where distinct (N = 5,000 `FP16 Extended`). Callout keys `[A]`–`[D]` in the external strip below the panels detail: **[A]** at N = 5,000 `exact`, `FP16 Extended` conventional `Recall@10 = 0.9950` (95% CI `[0.9850, 1.0000]`) and strict tie-aware `0.9950` ($\tau \le 10^{-6}$) reflect a single rank-10/rank-11 precision boundary on query `Q17` ($\Delta d = 1.665 \times 10^{-6}$ in `FP32`, reordered across the rank-10/rank-11 boundary under `FP16` quantization), which resolves to `1.0000*` at $\tau = 10^{-5}$; **[B]** at N = 1,001 `forced_hnsw`, all three representations achieve identical `20q = 0.9850 [0.9700, 1.0000]`, `19q = 0.9842 [0.9684, 1.0000]`, and tie-aware `0.9950`; **[C]** at N = 5,000 `forced_hnsw` (where `9/9` unforced planner profiles select HNSW), `FP32` achieves `20q = 0.9550 [0.9000, 1.0000]` (`Tie = 0.9850`) and `FP16 Extended` achieves `20q = 0.9600 [0.9050, 1.0000]` (`Tie = 0.9800` at 10^{-6}, `0.9850*` at 10^{-5}); and **[D]** at N = 10,000 `forced_hnsw` (where `9/9` unforced profiles select HNSW), excluding query `Q9` (which encounters a 28-passage duplicate-embedding-vector equality cluster where equality is established under `pgvector`'s vector equality operator `=` at verified rank-10 boundary cosine distance D = 0.44327880887538984, displayed as `0.443279`, without claiming identical original text or raw binary byte identity) raises `FP32 Extended` and `FP16 Extended` recall from `0.9350 [0.8450, 0.9950]` to `0.9737 [0.9368, 1.0000]` (`+0.0387`) and `FP32 Plain` from `0.9300 [0.8350, 0.9950]` to `0.9684 [0.9211, 1.0000]`, while tie-aware evaluation across all 20 queries reaches `0.9750` (`FP32 Extended`, `FP16 Extended`) and `0.9700` (`FP32 Plain`).

1. **Duplicate Embedding Vectors Equal Under `pgvector`'s Vector Equality Operator (=) on `Q9` (N = 10,000):** For Query `Q9` (*"difference between supervised and unsupervised machine learning"*, `query_id = 9`), the rank-10 boundary distance at N = 10,000 is D = 0.44327880887538984. The strictly closer set is B = {8105, 5314} (|B| = 2, with distances `0.432029` and `0.433520`; chunks `8105` and `5314` are distinct non-duplicate vectors with pairwise distance `0.3477`), while the boundary tie set T contains |T| = 28 chunks (`[166, 631, 684, 970, 1135, 2139, 3024, 3177, 3243, 3882, 4130, 4837, 5443, 6623, 7014, 7433, 7617, 8192, 8621, 8634, 8886, 9261, 9281, 9395, 9398, 9627, 9813, 9828]`) competing for r = 10 - 2 = 8 slots (`[Supplementary Material — Repository/DOI Placeholder]`). Read-only SQL verification (`SELECT (a.embedding = b.embedding), (a.embedding <=> b.embedding)` on `public.chunks_10k_fp32`) across all C(28, 2) = 378 pairs in T confirmed that **`378/378` pairs return `True` under `pgvector`'s vector equality operator (=) with cosine distance `0.0`**. This establishes that the 28 chunks are **duplicate embedding vectors equal under `pgvector`'s vector equality operator** (forming an **exact vector-equality cluster**, without independently establishing raw binary byte-for-byte storage identity or identical original corpus text). Without a secondary SQL tie-breaker key, the three modes return `8105` and `5314` followed by 8 different members of this 28-chunk exact vector-equality cluster:

   - **Ground-Truth `exact` (Seq Scan + Top-10 Heapsort) returned order:** `[8105, 5314, 3024, 3177, 166, 2139, 631, 970, 684, 1135]` (bounded heapsort emits tied chunks in heap-insertion/sift order rather than ascending `chunk_id`).
   - **`FP32 Extended` `forced_hnsw` returned order:** `[8105, 5314, 8634, 8621, 8192, 7617, 7433, 7014, 6623, 5443]`.
   - **`FP16 Extended` `forced_hnsw` returned order:** `[8105, 5314, 9828, 9813, 9627, 9395, 9398, 9281, 9261, 8886]`.

   Because the 8 tied chunks returned by `forced_hnsw` do not overlap the 8 tied chunks retained by the bounded heapsort in `exact` mode, ID-intersection `Recall@10` scores `Q9` as `2/10 =`

`0.20` (widening the 20-query bootstrap CI at N = 10,000 to `[0.8450, 0.9950]`, compared with `[0.9368, 1.0000]` across the remaining 19 queries), whereas equivalence-class `Tie-Aware Recall@10` ($\tau \in \{10^{-7}, 10^{-6}\}$) credits any 8 members of the 28-chunk exact vector-equality cluster (`10/10 = 1.00`, accounting for the entire `+0.0400` delta at N = 10,000).

2. **Separating Cluster Presence (11 / 2 / 1 Queries) from Actual Rank-10 Boundary Ties (6 Instances Across 5 Queries) and Genuine ϵ-Near-Ties (Q17):** A query can contain members of the recurring exact vector-equality cluster (4 chunks `[166, 631, 684, 970]` at N = 1,001, 12 at N = 5,000, 28 at N = 10,000) in its top 10 without experiencing a rank-10 boundary tie if all cluster members fall strictly inside ranks 1–9 (B). Specifically (`[Supplementary Material – Repository/DOI Placeholder]`):

- **Cluster presence in top-10:** Members of the exact vector-equality cluster appear in the top-10 of **11 queries at N = 1,001** (`Q2, Q3, Q5, Q8, Q9, Q12, Q13, Q14, Q16, Q17, Q18`), **2 queries at N = 5,000** (`Q3, Q9`), and **1 query at N = 10,000** (`Q9`).
- **Actual rank-10 boundary ties ($|T| > r$ at $\tau = 10^{-7}$ and 10^{-6}):** Exactly **6 query-scale instances across 5 unique queries** straddle rank 10: 3 at N = 1,001 (`Q5, Q13, Q16`), 2 at N = 5,000 (`Q3, Q9`), and 1 at N = 10,000 (`Q9`). For these 6 instances, `Tie-Aware Recall@10` at $\tau \le 10^{-6}$ evaluates equivalence-class membership over duplicate embedding vectors equal under `pgvector`'s vector equality operator rather than an approximate tolerance loosening.
- **Genuine ϵ-near-tie ($\tau = 10^{-5}$):** Relaxing τ from 10^{-6} to 10^{-5} adds exactly **1 instance** (`Q17` at N = 5,000, bringing the total to 7 query-scale instances across 6 unique queries), where chunks `1571` and `2675` are distinct vectors separated in FP32 cosine distance by $\Delta d = 1.665 \times 10^{-6}$ around D = 0.45295169. Crediting `Q17` at $\tau = 10^{-5}$ (`0.9950 -> 1.0000` in `exact` mode and `0.9800 -> 0.9850` in `forced_hnsw` for `fp16_extended`) represents an explicitly tolerant / optimistic ϵ-recall measure [23].

ACKNOWLEDGMENT

[Author Confirmation Required — Funding Statement: State external funding source(s) or confirm "This research received no external funding."]

AI-Use Disclosure (Provisional — Subject to Author Confirmation of Exact AI Systems/Models, Affected Manuscript Sections/Artifacts, and Functional Roles): In accordance with IEEE publication policies on artificial intelligence (AI)-assisted content, this disclosure is provisional pending explicit author confirmation of (i) the exact underlying AI systems and model versions invoked (note that platform or workspace names do not by themselves identify the underlying AI model version), (ii) the specific manuscript sections (`Sections I–VII, Appendix A`) and project artifacts (`Figures 1–3`, scripts, audit tables) each system materially helped generate or edit, and (iii) each system's specific role across coding, data analysis, statistical verification, independent review critique, and figure preparation: (1) **Google Gemini Web Tasks (`[Author Confirmation Required: specify exact underlying Gemini model version(s) and confirm materially edited manuscript sections, figure scripts, and IEEE formatting artifacts]`)**; (2) **OpenAI ChatGPT (`[Author Confirmation Required: specify exact ChatGPT model version(s) and confirm role in research planning, audit design, and revision specifications]`)**; (3) **Anthropic Claude (`[Author Confirmation Required: specify exact Claude model version(s) and confirm role in independent manuscript review critiques]`)**; and (4) **Antigravity IDE (`[Author Confirmation Required: specify exact underlying AI model version(s) invoked inside the Antigravity IDE workspace and confirm role in repository coding, read-only SQL checks, and 18-scale sweep / exact-cost validation execution]`)**. `[Author Confirmation Required – Verification & Responsibility Statement: Confirm before submission that all experimental benchmarks, database configurations, mathematical derivations, and final scientific claims were verified and approved by the author (Yash Pradhan), who takes full responsibility for the content of this publication.]`

REFERENCES

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[2] A. Kane and pgvector contributors, "pgvector: Open-source vector similarity search for Postgres — Documentation and type specifications (`vector`, `halfvec`, Version 0.8.4)," Jun. 30, 2026. [Online]. Available: https://github.com/pgvector/pgvector/blob/v0.8.4/README.md

[3] A. Kane and pgvector contributors, "pgvector 0.8.4 HNSW access method cost estimator implementation (`src/hnsw.c`, `hnswcostestimate`, Tag `v0.8.4`)," Jun. 30, 2026. [Online]. Available: https://github.com/pgvector/pgvector/blob/v0.8.4/src/hnsw.c

[4] PostgreSQL Global Development Group, *PostgreSQL 18.4 Documentation and Source Code: Chapter 66 (Database Physical Storage — TOAST) and Query Optimizer / Storage Internals (`src/backend/optimizer/path/costsize.c`, `src/backend/utils/adt/selfuncs.c`, `src/backend/optimizer/util/pathnode.c`, `src/backend/access/common/toast_internals.c`, `src/backend/access/heap/heapam.c`, `src/include/access/htup_details.h`, Tag `REL_18_4`)*, May 14, 2026. [Online]. Available: https://www.postgresql.org/docs/18/storage-toast.html ; https://git.postgresql.org/gitweb/?p=postgresql.git;a=blob;f=src/backend/optimizer/path/costsize.c;hb=REL_18_4 ; https://git.postgresql.org/gitweb/?p=postgresql.git;a=blob;f=src/backend/utils/adt/selfuncs.c;hb=REL_18_4 ; https://git.postgresql.org/gitweb/?p=postgresql.git;a=blob;f=src/backend/optimizer/util/pathnode.c;hb=REL_18_4

[5] Yu. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, Apr. 2020, doi: 10.1109/TPAMI.2018.2889473.

[6] W. Yang, T. Li, G. Fang, and H. Wei, "PASE: PostgreSQL ultra-high-dimensional approximate nearest neighbor search extension," in *Proc. 2020 ACM SIGMOD Int. Conf. Manage. Data (SIGMOD '20)*, Portland, OR, USA, Jun. 2020, pp. 2241–2253, doi: 10.1145/3318464.3386131.

[7] Y. Zhang, S. Liu, and J. Wang, "Are there fundamental limitations in supporting vector data management in relational databases? A case study of PostgreSQL," in *2024 IEEE 40th Int. Conf. Data Eng. (ICDE)*, Utrecht, Netherlands, May 2024, pp. 3640–3653, doi: 10.1109/ICDE60146.2024.00280.

[8] C. Wei, B. Wu, S. Wang, R. Lou, C. Zhan, F. Li, and Y. Cai, "AnalyticDB-V: A hybrid analytical engine towards query fusion for structured and unstructured data," *Proc. VLDB Endow.*, vol. 13, no. 12, pp. 3152–3165, Aug. 2020, doi: 10.14778/3415478.3415541.

[9] Q. Zhang, S. Xu, Q. Chen, G. Sui, J. Xie, Z. Cai, Y. Chen, Y. He, Y. Yang, F. Yang, M. Yang, and L. Zhou, "VBASE: Unifying online vector similarity search and relational queries via relaxed monotonicity," in *Proc. 17th USENIX Symp. Operating Syst. Design*

Implementation (OSDI '23), Boston, MA, USA, Jul. 2023, pp. 377–395, ISBN: 978-1-939133-34-2.

[10] H. Kim, C. Lim, H. An, R. Sen, and K. Park, "Exqutor: Extended query optimizer for vector-augmented analytical queries," in *2026 IEEE 42nd Int. Conf. Data Eng. (ICDE 2026)*, 2026, pp. 72–86, doi: 10.1109/ICDE65706.2026.00013.

[11] J. Shim, J. Oh, H. Roh, J. Do, and S.-W. Lee, "Turbocharging vector databases using modern SSDs," *Proc. VLDB Endow.*, vol. 18, no. 11, pp. 4710–4722, Jul. 2025, doi: 10.14778/3749646.3749724.

[12] H. Jégou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 33, no. 1, pp. 117–128, Jan. 2011, doi: 10.1109/TPAMI.2010.57.

[13] F. McSherry and M. Najork, "Computing information retrieval performance measures efficiently in the presence of tied scores," in *Advances in Information Retrieval (Proc. 30th Eur. Conf. IR Research, ECIR 2008)*, Lecture Notes in Computer Science, vol. 4956, Berlin, Heidelberg: Springer, 2008, pp. 414–421, doi: 10.1007/978-3-540-78646-7_38.

[14] T. Nguyen, M. Rosenberg, X. Song, J. Gao, S. Tiwary, R. Majumder, and L. Deng, "MS MARCO: A human generated MAchine Reading COmprehension dataset," in *Proc. Workshop Cognitive Comput.: Integrating Neural and Symbolic Approaches (CoCo@NIPS 2016)*, CEUR Workshop Proc., vol. 1773, 2016.

[15] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, "BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models," in *Proc. Neural Inf. Process. Syst. Track Datasets Benchmarks (NeurIPS Datasets and Benchmarks)*, 2021. [Online]. Available: https://datasets-benchmarks-proceedings.neurips.cc/paper_files/paper/2021/file/65b9eea6e1cc6bb9f0cd2a47751a186f-Paper-round2.pdf

[16] Z. Nussbaum, J. X. Morris, B. Duderstadt, and A. Mulyar, "Nomic Embed: Training a reproducible long context text embedder," *arXiv preprint arXiv:2402.01613*, 2024.

[17] W. J. Conover, *Practical Nonparametric Statistics*, 3rd ed. New York, NY, USA: John Wiley & Sons, 1999, ISBN: 978-0-471-16068-7.

[18] Y. Benjamini and Y. Hochberg, "Controlling the false discovery rate: A practical and powerful approach to multiple testing," *J. Roy. Statist. Soc. Ser. B (Methodol.)*, vol. 57, no. 1, pp. 289–300, 1995, doi: 10.1111/j.2517-6161.1995.tb02031.x.

[19] S. Holm, "A simple sequentially rejective multiple test procedure," *Scand. J. Statist.*, vol. 6, no. 2, pp. 65–70, 1979, doi: 10.2307/4615733.

[20] D. S. Kerby, "The simple difference formula: An approach to teaching nonparametric correlation," *Comprehensive Psychol.*, vol. 3, art. 1, 2014, doi: 10.2466/11.IT.3.1.

[21] G. Otochkin, "Vector data in Postgres: Size, TOAST, Filters and Performance," non-archival practitioner presentation at *PostgreSQL Conference Europe (PGConf.EU 2025)*, Riga, Latvia, Oct. 2025. [Online]. Available: https://www.postgresql.eu/events/pgconfeu2025/sessions/ ; https://www.youtube.com/watch?v=89Hw_4-PcRk

[22] L. F. Mackert and G. M. Lohman, "Index scans using a finite LRU buffer: A validated I/O model," *ACM Trans. Database Syst.*, vol. 14, no. 3, pp. 401–424, Sep. 1989, doi: 10.1145/68012.68015.

[23] M. Aumüller, E. Bernhardsson, and A. Faithfull, "ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms," *Inf. Syst.*, vol. 87, art. 101374, Jan. 2020, doi: 10.1016/j.is.2019.02.006.

[24] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann, "How good are query optimizers, really?," *Proc. VLDB Endow.*, vol. 9, no. 3, pp. 204–215, Nov. 2015, doi: 10.14778/2850583.2850594.

[25] *IEEE Standard for Floating-Point Arithmetic*, IEEE Std 754-2019 (Revision of IEEE Std 754-2008), Jul. 2019, pp. 1–82, doi: 10.1109/IEEESTD.2019.8766229.